Brightloop Subscription Management System — Overall Team Plan

Team size: 4 · **Hackathon:** Odoo Hackathon · **Problem:** Subscription Management System (Brightloop Software)

> **Revision 3.** The stack now matches UniBee's (Go + GoFrame, MySQL, React + TypeScript + Ant Design), and the billing core ports Kill Bill's infrastructure: typed and linked invoice items, compute-and-diff billing with safety bounds, dry runs, a database job queue, an audit log and a test clock. The full schema is in `db/migrations/000001_init.up.sql`, applied to **MySQL 8.0.46** and passing **62/62** checks in `db/test_constraints.sh`. Evidence for every external claim is in Part 3.

Contents

Part 1 — Team plan: goal, stack, repository layout, roles, contracts, database, decisions, timeline, Git, testing, scope, demo, evaluation, checklist

Part 2 — Odoo Subscriptions benchmark (revision 2; see the update note at its top)

Part 3 — Reverse engineering: UniBee and Kill Bill

Part 4 — Role guides: A Platform & Access · B Catalog & Pricing · C Sales & Subscriptions · D Billing, Payments & Reporting

Files delivered with this plan: `db/migrations/000001_init.up.sql` (schema), `db/test_constraints.sh` (62 checks), `reference/billingdate/` (Go period arithmetic with tests)

Part 1 — Team Plan

1. Goal and approach

Build one platform that runs Brightloop's subscription business end to end: products and plans, quotations, subscriptions, recurring invoices, taxes and discounts, payments and finance reporting. Following the hackathon guidelines, the priority is a **working, well-structured, database-driven system** over many unfinished features.

Each member owns a **vertical slice** (tables, Go logic, APIs, screens and tests), so everyone commits meaningfully and can present a complete flow. The schema is already written and validated, so Phase 0 is a walkthrough, not a design-from-scratch session.

---

2. Technology stack (matched to UniBee)

Layer	Choice	Matches	Why
Backend	**Go 1.22 + GoFrame v2.6.4**	UniBee `unibee-api` (Go 1.22.5, GoFrame v2.6.4)	Same framework as the reference; layered project layout
Database	**MySQL 8.0** (≥ 8.0.16 for enforced `CHECK`)	UniBee default; Kill Bill supported	Local relational DB, as the guidelines require
Migrations	**golang-migrate** SQL files	— (UniBee downloads upgrades from its platform API)	Reproducible offline
Money	`DECIMAL(12,2)` + `shopspring/decimal`	UniBee uses `shopspring/decimal`	Exact arithmetic; never `float64`
Locks and jobs	**MySQL** (`FOR UPDATE SKIP LOCKED`, `job_queue`)	Kill Bill's DB-backed queues	No Redis: one less service, same transaction as the data
Scheduler	**GoFrame** `gcron.AddSingleton`	UniBee	
Auth	`golang-jwt/jwt/v5`, `x/crypto/bcrypt`	UniBee	
PDF	`go-pdf/fpdf`	UniBee	Local library
Email	`net/smtp` → **MailHog**	— (UniBee uses SendGrid)	No third-party API
Frontend	**React 18 + TypeScript + Vite + Ant Design 5 + Zustand + axios + React Router 6**	UniBee admin and user portals	One design system
Run	**Docker Compose:** MySQL, MailHog, API, web	UniBee	One-command setup
Tests	Go `testing` + `testify`; `db/test_constraints.sh`; Vitest for frontend helpers	UniBee uses `testify`	

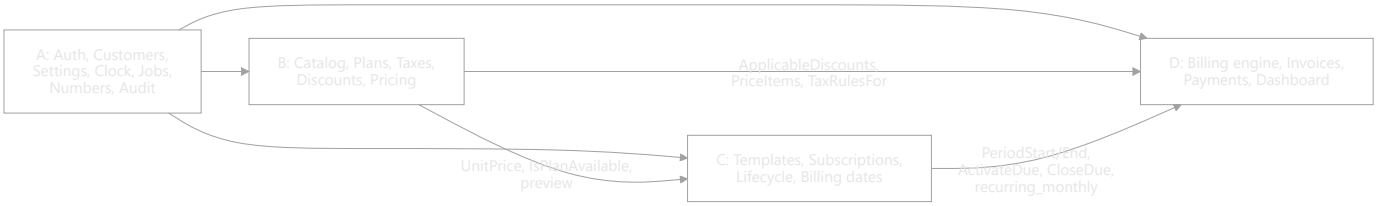
3. Repository layout

```
backend/
  main.go
  manifest/config/config.yaml
  api/{auth,admin,portal}/              -- request/response structs with g.Meta route + v: validation tags
  internal/
    cmd/ consts/ controller/            -- thin handlers
    logic/{auth,catalog,pricing,subscription,billing,payment,report,platform}/
    dao/ model/entity/                  -- `gf gen dao` (decimal typeMapping!)
    cronjob/                            -- gcron registrations
  db/migrations/000001_init.up.sql
  db/test_constraints.sh
frontend/src/{routes.tsx, requests/, stores/, components/, pages/}
reference/billingdate/                  -- C's period arithmetic, tested
docker-compose.yaml  README.md
```

4. Role summary

Member	Role	Owns
A	Platform & Access Lead (+ integration lead)	Auth, password rules and reset, roles, staff, customers, company settings, API conventions and error mapping, **clock + test clock, job queue, gapless numbering, audit log**, Docker Compose, migrations, UI kit, portal shell
B	Catalog & Pricing	Products, attributes and variants, plans, taxes, discounts, the **pricing service** that builds base, discount and tax items
C	Sales & Subscriptions	Templates, quotations, subscriptions and lines, the **state machine**, **billing-period date logic**, renewals, end-of-term close, add-ons, MRR figures, portal subscriptions
D	Billing, Payments & Reporting	**Billing engine** (diff, locks, safety bound, dry run), invoices and items, PDF, payments with idempotency, overdue reminders, dashboard, portal invoices, seed and performance tests

5. Module dependencies



Upstream members ship **stubs with the agreed Go signatures in Phase 1**.

6. Shared contracts

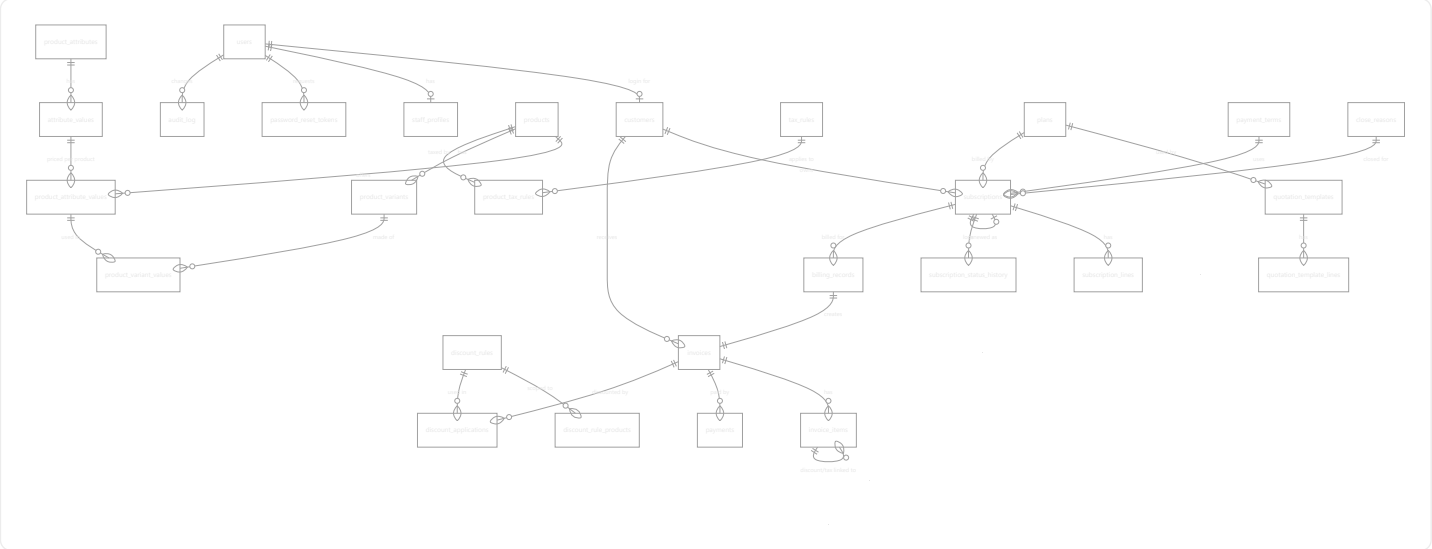
Provider	Contract	Consumers
A	`middleware.Auth`, `middleware.RequireRole`, `auth.CustomerID(ctx)`, `clock.Today(ctx)`, `sequence.Next(ctx, tx, name)`, `jobs.Enqueue(ctx, tx, ...)`, `audit.Log(ctx, tx, ...)`, `settings.Get(ctx)`, error mapping (1062 → `CONFLICT`, 3819 → `VALIDATION_ERROR`, 1213/1205 → retry)	All
B	`pricing.UnitPrice`, `pricing.TaxRulesFor`, `pricing.IsPlanAvailable`, `pricing.ApplicableDiscounts` (locks rules by ascending id), `pricing.PriceItems`, `POST /api/pricing/preview`	C, D
C	`subscription.Transition`, `billingdate.PeriodStart/PeriodEnd/NextPeriodOnOrAfter/MonthlyEquivalent`, `lifecycle.ActivateDue`, `lifecycle.CloseDue`	D, portal
D	`billing.Run(ctx, asOf, dryRun)`, `billing.Preview(ctx, subscriptionID)`, invoice balance	Scheduler, dashboard, portal

API conventions (A): success `{ "data", "meta" }`; error `{ "error": { "code", "message", "fields" } }`; resource routes; money as strings; dates `YYYY-MM-DD`; UTC everywhere; lists paginated (max 100).

---

7. Database overview

32 tables, 52 foreign keys, 38 check constraints, all in `db/migrations/000001_init.up.sql`.



Platform tables without business relationships: `company_settings`, `number_sequences`, `job_queue`, `billing_runs`.

Owner	Tables
A	`users`, `customers`, `staff_profiles`, `password_reset_tokens`, `company_settings`, `number_sequences`, `job_queue`, `audit_log`
B	`products`, `product_attributes`, `attribute_values`, `product_attribute_values`, `product_variants`, `product_variant_values`, `plans`, `tax_rules`, `product_tax_rules`, `discount_rules`, `discount_rule_products`
C	`payment_terms`, `close_reasons`, `quotation_templates`, `quotation_template_lines`, `subscriptions`, `subscription_lines`, `subscription_status_history`
D	`invoices`, `invoice_items`, `billing_records`, `discount_applications`, `payments`, `billing_runs`

Integrity rules enforced by MySQL (each covered by `db/test_constraints.sh`):

1. One account per email, case-insensitively (collation `utf8mb4_0900_ai_ci`).
2. One invoice per subscription per period: `billing_records UNIQUE (subscription_id, period_start)`.
3. Every confirmed or paid invoice has a number; numbers are unique; drafts have none.
4. "Paid" only when `amount_paid = total`; overpayment impossible.
5. Invoice items: discounts and taxes linked and correctly signed; base items never negative.
6. Discounts can't exceed `max_uses` or go below zero uses; percentages ⩽ 100; valid date ranges.
7. Only one default tax rule; at most one open renewal per subscription (generated-column unique markers, since MySQL has no partial indexes).
8. Closed subscriptions always have `closed_at`; expiry never before start; line amounts generated from quantity × price.
9. A payment form submitted twice records one payment (`idempotency_key`); a job enqueued twice exists once (`dedupe_key`).
10. Document numbers are gapless: a rolled-back increment is not consumed.
11. Invoices are never deleted; MySQL blocks it while items exist.
12. Money is `DECIMAL(12,2)` everywhere; foreign keys on every relationship (UniBee and Kill Bill both have none; we keep them deliberately).

---

8. Cross-module decisions (agree in kickoff, record in README)

#	Decision	Adopted rule	Affects
1	Plan price vs product lines	Plan price billed as its own recurring item; recurring product lines per period; one-time products on the first invoice only	B, C, D
2	Discount stacking	Single best eligible discount per invoice	B, D
3	Invoice structure	Kill Bill-style `invoice_items`: base items, plus discount and tax items **linked** to the item they apply to; append-only	B, D
4	Calculation and rounding	Discount per base item; tax per base item on its discounted amount; `decimal.Round(2)` per item; totals are sums of items	B, D
5	Duplicate protection	Expected-periods-minus-existing diff + unique period constraint + row lock with `SKIP LOCKED` + safety bound (31 periods per subscription per run)	C, D
6	Billing dates	Periods computed from `billing_anchor_date` with clamped month arithmetic (no `time.AddDate` for months)	C, D
7	New invoices	Draft without number; number on confirm from a gapless counter; optional auto-confirm setting	A, D
8	Payments	Partial allowed; overpayment rejected; only on confirmed invoices; idempotency key per form	A, D
9	Cancelling an invoice	Only without payments; releases discount uses; the period is not re-billed automatically	B, D
10	Pause	`paused` blocks billing; resume continues from the first period on or after today	C, D
11	Closing	Now (needs `can_close_early` before expiry) or at end of period (`scheduled_close_date`); `undo_close` before it happens; reason required	C, D
12	Renewal	Creates a linked renewal quotation; activating it closes the original as "Renewed"	C, D
13	Add-ons on active subscriptions	Effective from the next period, no proration	C, D
14	Nightly order	`ActivateDue` → `billing.Run` → `CloseDue`	C, D
15	Emails	Always via `job_queue` in the same transaction; retried with backoff	A, D
16	Test clock	Development only, forward-only, used by every date decision	A, C, D
17	MRR	Stored `recurring_monthly`; MRR, ARR, churn by reason on the dashboard	C, D
18	Who manages templates; who sees the dashboard	Decide admin-only vs admin + staff	A, C, D

Subscription lifecycle

`renew` creates a new linked subscription in `quotation`; `close` with `end_of_period` keeps the subscription active until `scheduled_close_date`.

Invoice lifecycle

`draft —confirm (number assigned)→ confirmed —(fully paid)→ paid`, and `draft/confirmed —cancel→ cancelled` (only if nothing has been paid). `payment_status` (`not_paid`, `partial`, `paid`) is generated alongside.

---

9. Timeline and checkpoints

Percentages of total hackathon time; for 24 hours, 10% ≈ 2.5 hours.

Phase	A — Platform	B — Catalog	C — Subscriptions	D — Billing
0. Kickoff (~10%) — all	Walk through schema and test script; repo, Compose	Pricing rules and item model	State machine and date rules	Billing algorithm and safety table
1. Foundation (~20%)	Compose, migrations, `gf gen dao` + decimal test, auth, middleware, error mapping, platform stubs, seed	Catalog/plan CRUD; **pricing stub**	Template/subscription CRUD; **`billingdate` package with tests**	Invoice/payment repositories; seed skeleton
✅ **Checkpoint 1**	Clean DB → migrate → `test_constraints.sh` passes → seed → login as all three roles, on every laptop			
2. Core (~35%)	Reset via jobs + MailHog, job worker, staff, app shell, customers, settings, test clock	Full pricing service + tests; taxes, discounts; admin screens; preview	Create from template; `Transition`; quotation, list, detail screens	`billing.Run`; invoice actions, PDF; payments; billing and invoice screens
✅ **Checkpoint 2**	A quote becomes an active subscription and produces a correct invoice with linked discount and tax items			
3. Integration (~15%)	Portal shell; audit customer isolation; audit-log screen	Locking order and limits with D	Lifecycle steps, renewals, end-of-term close, add-ons, portal	Scheduler, preview, dashboard, portal invoices, send, reminders
✅ **Checkpoint 3**	Signup → dashboard works on `main` with real data, using the test clock to cross a billing date			
4. Testing (~10%)	Role/ownership tests; clean-machine setup	Pricing tests	Transition and date tests	Billing retry/concurrency tests; 10k/100k performance run
5. Demo prep (~10%) — all	README	Demo catalog	Demo subscriptions	Demo invoices and payments

10. Git workflow

Protected `main`; branches `feature/<module>-<task>`; one reviewer per PR, ideally a dependent member (B ↔ D for discounts, C ↔ D for dates); rotate reviewers; commit messages `type(scope): summary` (UniBee enforces Conventional Commits in CI); merge small and often. **The schema is a shared file:** changes go in a new migration (`000002_...`) with its checks added to `test_constraints.sh`, reviewed by A.

Member	Branches
A	`feature/repo-setup`, `feature/migrations`, `feature/auth-core`, `feature/auth-password-reset`, `feature/platform-jobs-clock-audit`, `feature/staff-management`, `feature/customers-settings`, `feature/app-shell-ui-kit`, `feature/portal-shell`
B	`feature/catalog-products`, `feature/catalog-variants`, `feature/plans`, `feature/tax-rules`, `feature/discount-rules`, `feature/pricing-service`
C	`feature/quotation-templates`, `feature/subscriptions-crud`, `feature/billing-dates`, `feature/subscription-lifecycle`, `feature/lifecycle-jobs`, `feature/renewals-and-closing`, `feature/portal-subscriptions`
D	`feature/invoice-items`, `feature/billing-engine`, `feature/invoice-actions-pdf`, `feature/payments`, `feature/dashboard`, `feature/portal-invoices`, `feature/overdue-reminders`, `feature/seed-and-perf`

11. Testing plan

Layer	What	Owner
Database	`db/test_constraints.sh`: 62 checks, run in CI and before every migration merge	A (all extend it)
Go unit	Password rule, pricing and rounding, billing dates (`reference/billingdate`), state machine guards, billing diff and safety bound	A, B, C, D
Go integration (real MySQL)	Billing re-run, crash mid-run, two concurrent runs, discount last-use race, payment double submit, rolled-back number	D, B, A
API	Roles, customer isolation (404), error format for 1062/3819	A
Performance	10,000 subscriptions / 100,000 invoices; every page under 2 s; record the table	D
Setup	Fresh clone → README → running app	A

---

12. Scope

Must have: everything in the problem statement end to end: registration rules and reset, three roles, products with variants, plans, templates, the full lifecycle, renewals, automatic recurring invoices with no duplicates or gaps, automatic taxes and discounts, confirm/cancel/send/print, payments, overdue tracking, **MRR**, dashboard with filters, customer portal, under-two-second pages.

Should have (from Odoo, UniBee and Kill Bill; cheap and valuable): close reasons and churn, one-time products, add-ons on active subscriptions, close at end of term, upcoming-invoice preview, test clock, job queue with retries, audit log, overdue reminders, discount per-customer and per-cycle limits, auto-confirm setting.

Stretch: proration and scheduled plan changes (UniBee `subscription_pending_update`), credit notes and item adjustments (Kill Bill `ITEM_ADJ` / `CREDIT_ADJ`), customer credit balance (`CBA_ADJ`), CSV/XLSX export as a background task, optional products on templates, effective-dated blocking, login rate limiting.

Out of scope: payment gateways, multi-tenancy, usage billing, multi-currency, Redis.

---

13. Demo plan

Order	Presenter	Scene
1	A	Sign up (weak password rejected); reset email in MailHog; admin adds staff; restricted menus; audit-log entry
2	B	Product with Odoo variant (+560, computed); monthly and quarterly plans; tax rule; 10% seat discount with limits; preview showing linked discount and tax items
3	C	Quote from template; customer accepts in portal; activate; timeline; invalid action refused; close at end of period; renewal quotation
4	D	Upcoming-invoice preview; move the test clock; run billing; run again (0 created); confirm assigns the number; double-submitted payment recorded once; Paid; dashboard with MRR on 10k/100k data
5	All	Architecture, schema and test script; what we took from UniBee, Kill Bill and Odoo and what we left out; each member answers a question outside their module

---

14. Mapping to the evaluation criteria

Criterion	Evidence
Problem understanding	Every scene maps to a module and a demo step; three reference systems studied with cited evidence
Logic and correctness	State machine, pricing service, compute-and-diff billing, DB-enforced invariants
Coding standards	GoFrame layered layout, shared conventions, reviews, Conventional Commits
Modularity	Four vertical modules with Go contracts
Database design	32 tables, 52 FKs, 38 checks, validated on MySQL 8.0.46 with 62 tests
Backend and APIs	Resource routes, `g.Meta` structs with validation, role checks, error mapping
Frontend	Ant Design system, status tags, confirmations, loading/empty/error states
Performance	Billing and dashboard queries measured at 8–99 ms on 10k/100k data
Scalability	Keyset batches, `SKIP LOCKED` workers, stored aggregates, paginated APIs
Security	bcrypt, hashed reset tokens, httpOnly JWT, backend RBAC, customer isolation
Usability	Clear validation, only valid actions shown, previews before billing
Debugging	`billing_runs`, `billing_records.inputs`, audit log, job errors, status history
Attention to detail	Month-end dates, rounding, decimal mapping, RE2 regex, bcrypt limit, gapless numbers
Teamwork	Vertical ownership, rotating reviews, shared schema process, everyone presents

---

15. Final submission checklist

- [ ] `docker compose up` → migrate → `db/test_constraints.sh` 62/62 → seed → app runs from the README on a clean machine
- [ ] Registration and password rules identical on client and server; reset via MailHog
- [ ] Admin/staff/customer separation on the backend; customer isolation returns 404
- [ ] Products, variants, plans, taxes, discounts and templates manageable by the admin
- [ ] Lifecycle, renewals and end-of-term close follow plan rules; invalid transitions blocked
- [ ] Recurring invoices: no duplicates on re-run or concurrent runs, no skipped periods, safety bound works
- [ ] Linked discount and tax items correct to the cent; totals equal the preview
- [ ] Numbers assigned on confirm, gapless; payments idempotent; paid/partial/outstanding/overdue visible
- [ ] Dashboard with MRR, filters and summary; timings recorded under 2 s
- [ ] No `float64` money anywhere; no `time.Now()` in business logic (use `clock.Today`)
- [ ] Claims about UniBee, Kill Bill and Odoo stated as in Parts 2–3, no stronger
- [ ] All four members have meaningful commits and can explain the whole system

---

Part 2 — Odoo Subscriptions Benchmark

Purpose: compare our design with Odoo, the product this hackathon is named after, adopt what makes our system more correct, and be able to explain to reviewers where we follow Odoo and where we deliberately don't.

> **Update (revision 3).** After studying Kill Bill, two decisions below were superseded: discounts are now `discount` items linked to each base item, and taxes are `tax` items linked the same way, all in one `invoice_items` table. This replaces the per-tax-group discount lines (O12) and the `invoice_line_taxes` table (O8). The database is now MySQL 8.0. Everything else here still stands; see the UniBee and Kill Bill findings.

---

1. How this was investigated (and its limits)

Source	What it gave us	Confidence
Odoo 18.0 Community source on GitHub (`odoo/odoo`, branch `18.0`)	Data models Odoo Subscriptions is built on: quotation templates, sales orders, product attributes and variant pricing, taxes, payment terms, invoices, payments, discount programs	**Observed.** Field definitions read directly from the files cited below
Odoo Subscriptions app itself (`sale_subscription`)	Recurring plans, subscription states, renewals, MRR, close reasons, recurring invoicing	**Not inspected.** It is an Enterprise (closed-source) module: `addons/sale_subscription` does not exist in the public repo (HTTP 404). Statements about it below come from general product knowledge and are marked *unverified*
odoo.com/app/subscriptions	Marketing page	**Not reached.** odoo.com is outside this environment's allowed network, so the page could not be loaded

The reverse-engineering toolkit (REA) was not used: its own guidance says to use ordinary repository tools when the source is available, which is the case for everything we could inspect, and the closed Enterprise module could not be obtained.

Before the presentation, someone should verify the *unverified* items on a free Odoo trial (Subscriptions app) so no one states them as fact.

---

2. Findings ledger

Observed in Odoo 18 Community source

#	Finding	Evidence (file → field)	Our decision
O1	Quotation templates carry lines, optional products, a validity in days, terms and conditions, and online signature/payment options	`sale_management/models/sale_order_template.py` → `number_of_days`, `note`, `sale_order_template_line_ids`, `sale_order_template_option_ids`, `require_signature`, `require_payment`	**Adopted:** `validity_days` and `terms` on templates. Optional products, signature and online payment → not adopted (see §4)
O2	Without a template, quotation validity comes from a company-wide default	`sale/models/sale_order.py` → `_compute_validity_date` uses `company_id.quotation_validity_days`	**Adopted:** `company_settings.default_quotation_validity_days` (A)
O3	Sales documents use states Quotation → Quotation Sent → Sales Order → Cancelled	`sale/models/sale_order.py` → `SALE_ORDER_STATE`	**Kept ours:** the problem statement prescribes Draft → Quotation → Confirmed → Active → Closed; Odoo's sales states map onto our first three
O4	A variant's extra price lives on the **product × attribute value** pair, not on the global value or the variant	`product/models/product_template_attribute_value.py` → `price_extra`; `ptav_active`	**Adopted:** new `product_attribute_values` table with `price_extra`; variant price = base + sum of extras; used values archived, not deleted (B)
O5	Attributes can create variants instantly, dynamically or never	`product/models/product_attribute.py` → `create_variant` (`always`, `dynamic`, `no_variant`)	**Simplified:** we always create explicit variants; noted as a known simplification
O6	Invoices have a document state **and** a separate payment state	`account/models/account_move.py` → `state` (`draft`, `posted`, `cancel`) and `payment_state` (`not_paid`, `partial`, `in_payment`, `paid`, `reversed`, …)	**Adopted:** our `status` follows the problem; new generated `payment_status` (`not_paid`, `partial`, `paid`) (D)
O7	Invoice numbers are assigned **when posted**, not when created; drafts are named "/", and Odoo tracks sequence gaps	`account/models/account_move.py` → `_compute_name` (only names when `state != 'draft'`), `made_sequence_gap`	**Adopted:** number assigned on confirm; DB check that confirmed invoices have a number (D)
O8	A line can carry several taxes; taxes can be percent, fixed, group, or price-included	`account/models/account_move_line.py` → `tax_ids` (many-to-many); `account/models/account_tax.py` → `amount_type`, `price_include`	**Adopted:** multiple taxes per line, snapshotted in `invoice_line_taxes` (B, D). Percent and tax-exclusive only for the MVP
O9	Payment terms are made of lines (percent or fixed, with delay rules such as "days after end of month")	`account/models/account_payment_term.py` → `value`, `value_amount`, `delay_type`, `nb_days`	**Simplified:** one `due_days` per term; installments not needed for the problem
O10	Payments record amount, date, memo and method, and	`account/models/account_payment.py` → `amount`, `date`, `memo`, `payment_method_line_id`, `state`	**Confirms** our payment fields (method, amount, date, note)

#	Finding	Evidence (file → field)	Our decision
	have their own states		
O11	Discount programs have validity dates, a usage limit, conditions (min quantity, min amount, products/categories) and rewards (percent or fixed, order or specific products, max amount)	`loyalty/models/loyalty_program.py` → `date_from`, `date_to`, `limit_usage`, `max_usage`, `trigger`; `loyalty_rule.py` → `minimum_qty`, `minimum_amount`, `product_ids`; `loyalty_reward.py` → `discount_mode`, `discount_applicability`, `discount_max_amount`	**Confirms** our discount rule design; **adopted** `max_discount_amount` (B)
O12	Discounts are applied as **separate reward lines**, split by tax group	`sale_loyalty/models/sale_order.py` → reward lines (`is_reward_line`), `discountable_per_tax`	**Adopted:** discounts become their own negative invoice lines, one per tax group (B, D)

From product knowledge — *unverified, check on a trial*

#	Finding	Our decision
K1	Products are flagged as recurring, and one-time items can sit on the same subscription (e.g. setup fees)	**Adopted:** `products.is_recurring`; one-time lines billed on the first invoice only (B, D)
K2	Renewal produces a **new renewal quotation** linked to the original, rather than editing the running subscription	**Adopted:** `renew` creates a linked subscription (`renewed_from_id`); the original closes as "Renewed" when the renewal activates (C)
K3	Subscriptions store a monthly recurring amount, and reporting leads with **MRR**, ARR and churn	**Adopted and promoted to core:** `recurring_monthly` on subscriptions; MRR, ARR and churn on the dashboard (C, D). The problem itself says "nobody can say how much recurring revenue the company actually has"
K4	Closing a subscription records a **close reason**, which drives churn analysis	**Adopted:** `close_reasons` table, reason required on close/cancel (C)
K5	Recurring invoices are generated by a scheduled job from each subscription's next invoice date, and can be posted automatically	**Confirms** our billing engine; **adopted** optional `auto_confirm_recurring_invoices` setting (A, D)
K6	Mid-term changes (upsells) are handled with prorated orders	**Partly adopted:** add/remove add-ons on active subscriptions, effective next period, no proration (C). Proration → stretch
K7	Customers can pay by saved card and be charged automatically, with retries and automatic closing after unpaid days	**Not adopted:** requires a payment gateway (third-party dependency the guidelines discourage)
K8	Customer portal for viewing, renewing, closing and paying subscriptions	**Confirms** our portal scope (pay online excluded, as K7)

3. Summary of plan changes

Area	Change	Owner
Variants	`price_extra` moved to new `product_attribute_values` (product × value); variant price computed	B
Products	`is_recurring` flag; one-time items billed once	B, D
Discounts	`max_discount_amount` cap; discount shown as separate negative lines split by tax group	B, D
Taxes	Several taxes per line, snapshotted in `invoice_line_taxes`	B, D
Templates	`terms`; default validity from company settings	C, A
Renewal	Creates a linked renewal subscription; original closes as "Renewed"	C
Closing	`close_reasons` table, reason required; churn reporting	C, D
MRR	`recurring_total` / `recurring_monthly` on subscriptions; MRR/ARR/churn on the dashboard (now **core**)	C, D
Add-ons	Add/remove lines on active subscriptions, effective next period	C
Invoices	Number assigned on confirm; generated `payment_status`; line types; period in line descriptions	D
Settings	New `company_settings` row: company details, currency, invoice prefix, default validity, auto-confirm	A

---

4. Deliberately not adopted (and why)

Odoo feature	Reason
Online card payments, saved tokens, automatic charging and payment retries	Needs a payment provider; guidelines discourage third-party dependencies. Payments are recorded manually as the problem describes
Electronic signature on quotations	Not in the problem; adds complexity without demo value
Optional products on templates	Nice for upselling, not required; candidate stretch
Prorated upsells	Correct proration is subtle; add-ons apply from the next period instead
Credit notes / invoice reversal	We cancel unpaid invoices only; credit notes are a stretch item
Installment payment terms, early-payment discounts	Not required; `due_days` covers the problem
Pricelists, multi-currency, multi-company, fiscal positions	Out of scope for Brightloop
Tax-included prices, fixed and group taxes	MVP supports percent, tax-exclusive; schema can extend
Dynamic / no-variant attributes	Explicit variants are simpler and sufficient
Sales-document state separate from subscription state	The problem prescribes one lifecycle; one enum keeps it explainable

---

5. How to use this in the presentation

- Open with: "We studied how Odoo models the same problem and adopted what makes billing correct: invoice numbers on confirmation, discount lines per tax group, price extras per product and value, renewals as linked records, and MRR."
- Be explicit about what was **observed in source** versus **assumed about the Enterprise app**.
- Use §4 to show judgement: every omission has a reason tied to the problem statement or the guidelines.

Part 3 — Reverse Engineering: UniBee and Kill Bill

Purpose: study two open-source billing systems, match our stack to UniBee's, adopt Kill Bill's billing infrastructure where it makes billing more reliable, and record the evidence for every claim so the team can defend it in the presentation.

---

1. Method and limits

Item	Detail
Sources read	`github.com/UniBee-Billing/unibee` (deployment repo) and its code repos `unibee-api`, `unibee-admin-portal`, `unibee-user-portal`; `github.com/killbill/killbill` (master, version `0.25.7-SNAPSHOT`) and enums from `killbill/killbill-api`; GoFrame `v2.6.4`; `shopspring/decimal v1.3.1`; `golang/crypto`
How	Static reading of the source. Both projects are ordinary source repositories, so normal repository tools were used rather than binary reverse-engineering tooling
Our own verification	Our MySQL schema was applied to **MySQL 8.0.46** (32 tables, 52 foreign keys, 38 check constraints) and passed **62/62** behavioural checks (`db/test_constraints.sh`); concurrency, deletion and performance were tested; Go-specific claims were checked with **Go 1.22.2**
Not done	Neither UniBee nor Kill Bill was run. Kill Bill's Java version is set in its parent POM, which wasn't inspected. The GoFrame decimal mapping is taken from GoFrame's source and documentation but must be confirmed by A's Phase 1 round-trip test. Timings come from this sandbox and must be re-measured on the team's machines

2. Stack comparison and our choice

Layer	UniBee (observed)	Kill Bill (observed)	Ours	Why
Language / framework	Go 1.22.5, GoFrame `v2.6.4` (`unibee-api/go.mod`)	Java; Maven modules; Guice, Jetty, Jersey (module POMs)	**Go 1.22 + GoFrame v2.6.4**	Matches UniBee exactly; Kill Bill's ideas are ported, not its Java stack
Database	MySQL 8.0.37 in Docker Compose; a PostgreSQL driver is also registered (`internal/driver/pgsql`)	MySQL, MariaDB and PostgreSQL (drivers and embedded-DB test modules); DDL is MySQL-first	**MySQL 8.0**	Common to both; allowed by the guidelines
Money	`bigint` cents (`amount, cent`); tax as `1000 = 10%`	`numeric(15,9)`	**`DECIMAL(12,2)` + `shopspring/decimal`**	Exact; UniBee already uses `shopspring/decimal` in Go
Locks / queues	Redis 6 (`SET NX EX` locks, `go-redismq`)	Database tables (`notifications`, `bus_events`) and a DB lock service	**MySQL only** (`FOR UPDATE SKIP LOCKED`, `job_queue`)	One less service; locks and jobs share the business transaction (see U7)
Scheduler	GoFrame `gcron.AddSingleton`	Notification queues (`next-billing-date-queue`)	**GoFrame `gcron.AddSingleton`**	Matches UniBee
Auth	`golang-jwt/jwt/v5`	—	**`golang-jwt/jwt/v5` + `x/crypto/bcrypt`**	Matches UniBee
PDF	`go-pdf/fpdf`	—	**`go-pdf/fpdf`**	Local library, no service
Email	SendGrid SDK	—	**`net/smtp` → MailHog**	Guidelines discourage third-party APIs
Migrations	Upgrades fetched from UniBee's platform API at start-up (`logic/dbupgrade/upgrade.go`)	DDL files	**golang-migrate SQL files**	Reproducible offline, as the guidelines require
Frontend	React 18, TypeScript, Vite, Ant Design 5, Zustand, axios, React Router 6 (both portals; user portal adds Tailwind)	Separate admin UI (not inspected)	**React 18 + TypeScript + Vite + Ant Design 5 + Zustand + axios + React Router 6**	Matches UniBee; one design system (no Tailwind)
Deployment	Docker Compose + Nginx	—	**Docker Compose** (MySQL, MailHog, API, web)	Matches UniBee; one-command setup

3. UniBee findings

#	Finding	Evidence	Decision
U1	The linked repo holds deployment files only; code lives in three other repos	`unibee/README.md`, `docker-compose.yaml`	—
U2	39 tables, **no foreign keys**	`mysql/structure.sql` (`FOREIGN KEY` count: 0)	**Not adopted.** We keep 52 foreign keys; the guidelines value them
U3	Invoice lines are stored as JSON text	`invoice.lines text COMMENT 'lines( json)'`	**Not adopted.** We use `invoice_items` rows
U4	Billing: two singleton jobs every 10 s pick up to 10 subscriptions whose `dunning_time` has passed	`internal/cronjob/cronjob.go`, `internal/cronjob/sub/subscription.go`	**Adopted** the polling + `AddSingleton` pattern
U5	Whether to invoice is decided in code from the latest invoice's status and period; there is no unique key per subscription period	`logic/subscription/billingcycle/cycle/cycle.go` (`needInvoiceGenerate`); invoice unique key is only `unique_id`	**Improved on:** `billing_records UNIQUE (subscription_id, period_start)`
U6	Per-subscription lock: Redis `SET key 1 NX EX 60`	`utility/jedis.go` `TryLock`	**Replaced** by a MySQL row lock
U7	The lock is released with `DEL` without checking its value	`utility/jedis.go` `ReleaseLock`	Reason for U6: a worker that outlives the 60 s expiry can delete another worker's lock
U8	Subscriptions carry `billing_cycle_anchor`, `current_period_start/end`, `cancel_at_period_end`, `trial_end`	`subscription` table	**Adopted** anchor and end-of-term close
U9	Per-subscription **test clock**, forward-only, capped, excluded from production billing	`billingcycle/testclock/testclock.go`, `cronjob/sub/subscription.go`	**Adopted** as a global development-only, forward-only clock
U10	Plans: `interval_unit` (day/week/month/year) + `interval_count`; main / recurring add-on / one-time add-on types	`plan` table	**Adopted** `billing_interval_count` and one-time products
U11	Discount codes: percentage or fixed, `user_limit`, `subscription_limit`, `cycle_limit`, dates; usage rows with status `1-normal, 2-rollback`	`merchant_discount_code`, `merchant_user_discount_code`	**Adopted** per-customer and per-subscription-cycle limits; reversal on cancel
U12	Scheduled plan changes with proration	`subscription_pending_update`	Stretch
U13	Idempotency via `unique_id` unique keys on invoice, payment, timeline	`structure.sql`	**Adopted** for payments
U14	Operation log and async export tasks	`merchant_operation_log`, `merchant_batch_task`	**Adopted** audit log; exports are stretch
U15	API structs with `g.Meta` route tags; admin (`api/merchant`) and customer (`api/user`) split; updates often `POST .../update`	`unibee-api/api/`	**Adopted** structs and the admin/portal split; we keep resource-style routes
U16	Stripe, SendGrid, VAT services, multi-merchant (`merchant_id` everywhere)	`go.mod`, `structure.sql`	**Not adopted** (third-party, out of scope)

4. Kill Bill findings

#	Finding	Evidence	Decision
K1	Every invoice amount is a row in `invoice_items` with a `type`, a service period (`start_date`, `end_date`) and `linked_item_id`; items have only `created_*` columns (append-only)	`invoice/src/main/resources/.../invoice/ddl.sql`	**Adopted:** `invoice_items` with `recurring`, `fixed`, `discount`, `tax`, linked and append-only
K2	Item types: `EXTERNAL_CHARGE, FIXED, RECURRING, REPAIR_ADJ, CBA_ADJ, CREDIT_ADJ, ITEM_ADJ, USAGE, TAX, PARENT_SUMMARY`	`killbill-api` `InvoiceItemType.java`	Subset adopted; adjustments, credits and usage are stretch / out of scope
K3	Invoice statuses `DRAFT, COMMITTED, VOID`	`killbill-api` `InvoiceStatus.java`	Map to our draft / confirmed / cancelled; "paid" from the problem statement
K4	The generator builds "ALL items since beginning of time", merges them with existing items and writes only the difference	`generator/FixedAndRecurringInvoiceItemGenerator.java` (`generateItems`)	**Adopted** as expected periods minus `billing_records`
K5	Safety bounds: duplicate-item sanity check on by default; max **15** items per subscription per day	same file (`safetyBounds`); `util/.../InvoiceConfig.java` defaults	**Adopted:** max 31 periods per subscription per run
K6	Global lock per account around invoicing and payments; not taken for dry runs	`InvoiceDispatcher.java` (`ACCNT_INV_PAY`)	**Adapted:** row locks on the subscription (billing) and invoice (payments); discount rows locked in id order
K7	Dry-run invoices	`InvoiceDispatcher.java` (`DryRunArguments`)	**Adopted:** upcoming-invoice preview, dry-run billing runs
K8	DB-backed queues: `notifications`, `bus_events` with `processing_state` (default `AVAILABLE`), `processing_owner`, `processing_available_date`, `error_count`, `effective_date`	`util/src/main/resources/.../util/ddl.sql`	**Adopted:** `job_queue`
K9	`audit_log` (`table_name`, `target_record_id`, `change_type`, `reason_code`, `comments`) and a `*_history` table per entity	same `ddl.sql`; module DDLs	**Adopted** `audit_log`; history only for subscription status
K10	Test-mode `ClockMock` behind a test endpoint, with a warning that moving the clock back and forth may cause weird side effects	`jaxrs/.../resources/TestResource.java`	**Adopted** (forward-only)
K11	`payments.external_key` unique per tenant	`payment/.../ddl.sql` (`payments_key`)	**Adopted:** `payments.idempotency_key`
K12	Pause/blocking via `blocking_states` (`block_change`, `block_entitlement`, `block_billing`, `effective_date`)	`entitlement/.../ddl.sql`	Our `paused` status blocks billing; effective-dated blocking is stretch

#	Finding	Evidence	Decision
K13	Cancellation policies `IMMEDIATE`, `END_OF_TERM`; billing policies `START_OF_TERM`, `END_OF_TERM`, `IMMEDIATE`; `UNCANCEL` event	`killbill-api` `Entitlement.java`, `BillingActionPolicy.java`; `subscription/.../ApiEventType.java`	**Adopted:** close now / at end of period, `undo_close`
K14	Bill cycle day per account; alignment `ACCOUNT, BUNDLE, SUBSCRIPTION`	`account/.../ddl.sql` (`billing_cycle_day_local`); `BillingAlignment.java`	**Adopted** subscription alignment via `billing_anchor_date`
K15	Snapshot of billing events per invoice	`invoice_billing_events`	**Adopted:** `billing_records.inputs` JSON
K16	Subscriptions are event-sourced (`subscription_events`: `CREATE, CHANGE, CANCEL, UNDO_CHANGE, UNCANCEL`, `PHASE`, `BCD_UPDATE`, ...)	`subscription/.../ddl.sql`, `ApiEventType.java`, `SubscriptionBaseEvent.java`	**Not adopted:** status + history table is enough for this problem and easier to explain
K17	No foreign keys in any DDL; internal `record_id` plus external UUID `id`; invoice number = `record_id`	DDL files (count 0); `InvoiceSqlDao.sql.stg`	**Not adopted:** we keep FKs and gapless numbers on confirm
K18	Multi-tenancy, plugins, usage billing, parent/child invoices, credit balance (`CBA_ADJ`)	module list, DDL	Out of scope / stretch

5. Verified technical facts that changed our plan

#	Fact	How verified	Effect
V1	Go's `regexp` (RE2) rejects lookaheads: `invalid or unsupported Perl syntax: (?=`	Compiled with Go 1.22.2	Password rule checked as separate patterns (A)
V2	GoFrame's `password3` rule is 6–18 characters and requires a digit	`gvalid/internal/builtin/builtin_password3.go`	Not used; custom rule (A)
V3	bcrypt returns `ErrPasswordTooLong` above 72 bytes	`golang/crypto` `bcrypt/bcrypt.go`	Max 72 bytes (A)
V4	`gf gen dao` maps `decimal` to `float64` by default; `typeMapping` can map it to `decimal.Decimal`	`cmd/gf/internal/cmd/gendao/gendao.go` (`defaultTypeMapping`, example config)	Mandatory config + round-trip test (A)
V5	GoFrame `LockUpdate()` emits only `FOR UPDATE`	`database/gdb/gdb_model_lock.go`	`SKIP LOCKED` in raw SQL (D)
V6	`decimal.Round` documents `5.45 → 5.5` (half away from zero)	`shopspring/decimal v1.3.1` source	Rounding rule stated precisely (B)
V7	`time.AddDate` normalises: 31 Jan + 1 month = 3 Mar (2026)	Go test in `reference/billingdate`	Clamped month arithmetic (C)
V8	PostgreSQL sequences don't roll back; a MySQL counter row does	MySQL test (counter unchanged after rollback)	Gapless numbers via `number_sequences` (A, D); corrects our earlier "no gaps" claim
V9	MySQL has no partial indexes or case-insensitive text type	MySQL 8.0 behaviour; generated-column workarounds tested	Single default tax, one open renewal, case-insensitive email (A, B, C)
V10	MySQL blocks deleting a parent item while linked items reference it	Deletion test (error 1451)	Invoices are never deleted; no cascades on invoice children (D)
V11	`SKIP LOCKED` returns immediately without the locked row	Two sessions: `[11,12]` returned in 0.01 s while 10 was locked	Basis of concurrent billing (D)
V12	Dashboard and billing queries take 8–99 ms on 10,000 subscriptions / 100,000 invoices	Seeded MySQL timings	Two-second target credible; re-measure in Phase 4 (D)

---

6. How to use this in the presentation

- "We matched UniBee's stack (Go, GoFrame, MySQL, React and Ant Design) and ported Kill Bill's billing infrastructure: typed and linked invoice items, compute-and-diff billing with safety bounds, dry runs, a database job queue, an audit log and a test clock."
- "Where we deliberately differ, it's to follow the guidelines: foreign keys everywhere, no Redis, no third-party email or payments, no JSON business data, and a database constraint that makes double billing impossible."
- Every claim above has a source file; say "observed in the source" or "verified on MySQL 8.0", never more.

Part 4 — Role Guides

The sections below are the detailed guides for each member.

Member A — Platform & Access Lead

Project: Brightloop Subscription Management System **Stack:** Go 1.22 + GoFrame v2.6.4 · MySQL 8.0 · React 18 + TypeScript + Vite + Ant Design 5 **Modules owned:** Authentication, users and roles, staff management, customers, company settings, and the shared platform pieces: API conventions, middleware, clock, job queue, audit log, document numbering, shared UI kit **Additional role:** Integration lead (repo, Docker Compose, migrations, README, merge coordination)

---

1. Your mission

Everyone else builds on your work, so it must be stable by the end of Phase 1: the team logs in as admin, staff and customer, every API is protected by your middleware, and migrations run on every laptop. You also own the platform services the billing engine depends on: the **clock** (with a test clock

for the demo), the **job queue** for reliable emails, **gapless document numbers**, and the **audit log**. And you guarantee that **a customer can never see another customer's data**.

---

2. Database tables you own

These statements are copied from `db/migrations/000001_init.up.sql`, which has been applied to MySQL 8.0.46 and passes all 62 checks in `db/test_constraints.sh`.

sql

```sql
CREATE TABLE users (
    id            BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    email         VARCHAR(254) NOT NULL COLLATE utf8mb4_0900_ai_ci, -- case-insensitive collation → UNIQUE is case-insens
    password_hash VARCHAR(255) NOT NULL,                            -- bcrypt
    role          ENUM('admin', 'staff', 'customer') NOT NULL,
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
    created_at    DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at    DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_users_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE customers (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id         BIGINT UNSIGNED NULL,                -- NULL if staff created the customer without a login
    name            VARCHAR(150) NOT NULL,
    company_name    VARCHAR(150) NULL,
    phone           VARCHAR(30) NULL,
    billing_address TEXT NULL,
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_customers_user (user_id),
    KEY idx_customers_name (name),
    CONSTRAINT fk_customers_user FOREIGN KEY (user_id) REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE staff_profiles (
    id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id    BIGINT UNSIGNED NOT NULL,
    full_name  VARCHAR(150) NOT NULL,
    job_title  VARCHAR(100) NULL,
    created_by BIGINT UNSIGNED NOT NULL,                -- must be an admin (checked in the service layer)
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_staff_user (user_id),
    CONSTRAINT fk_staff_user       FOREIGN KEY (user_id)    REFERENCES users (id),
    CONSTRAINT fk_staff_created_by FOREIGN KEY (created_by) REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE password_reset_tokens (
    id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id    BIGINT UNSIGNED NOT NULL,
    token_hash CHAR(64) NOT NULL,                       -- SHA-256 hex of the emailed token
    expires_at DATETIME(3) NOT NULL,
    used_at    DATETIME(3) NULL,                        -- single use
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_reset_token_hash (token_hash),
    KEY idx_reset_tokens_user (user_id),
    CONSTRAINT fk_reset_user FOREIGN KEY (user_id) REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE company_settings (
    id                              TINYINT UNSIGNED NOT NULL DEFAULT 1,
    company_name                    VARCHAR(150) NOT NULL DEFAULT 'Brightloop Software',
    address                         TEXT NULL,
    currency_code                   CHAR(3) NOT NULL DEFAULT 'INR',
    invoice_prefix                  VARCHAR(10) NOT NULL DEFAULT 'INV',
    default_quotation_validity_days INT UNSIGNED NOT NULL DEFAULT 30,
    auto_confirm_recurring_invoices BOOLEAN NOT NULL DEFAULT FALSE,
    test_clock_date                 DATE NULL,          -- honoured only when APP_ENV=development; forward-only
    updated_at                      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
```

```sql
    CONSTRAINT chk_company_settings_single_row CHECK (id = 1)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Gapless document numbers (MySQL has no sequences; a locked counter row also rolls back cleanly)
CREATE TABLE number_sequences (
    name       VARCHAR(40) NOT NULL,                    -- e.g. 'subscription-2026', 'invoice-2026'
    next_value BIGINT UNSIGNED NOT NULL DEFAULT 1,
    PRIMARY KEY (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- DB-backed job queue / transactional outbox (Kill Bill notifications pattern)
CREATE TABLE job_queue (
    id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    queue_name   VARCHAR(64) NOT NULL,                    -- 'email', 'overdue-reminder'
    dedupe_key   VARCHAR(150) NULL,                       -- e.g. 'invoice-sent:42'; NULLs allowed many times
    payload      JSON NOT NULL,
    effective_at DATETIME(3) NOT NULL,                    -- run no earlier than this
    state        ENUM('available', 'processing', 'done', 'failed') NOT NULL DEFAULT 'available',
    attempts     INT UNSIGNED NOT NULL DEFAULT 0,
    max_attempts INT UNSIGNED NOT NULL DEFAULT 5,
    locked_by    VARCHAR(64) NULL,
    locked_until DATETIME(3) NULL,
    last_error   TEXT NULL,
    created_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    processed_at DATETIME(3) NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_jobs_dedupe (dedupe_key),
    KEY idx_jobs_ready (state, effective_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Audit trail (Kill Bill audit_log / UniBee merchant_operation_log)
CREATE TABLE audit_log (
    id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    table_name VARCHAR(64) NOT NULL,
    record_id  BIGINT UNSIGNED NOT NULL,
    action     VARCHAR(40) NOT NULL,                      -- 'create', 'update', 'confirm', 'cancel', ...
    changed_by BIGINT UNSIGNED NULL,                      -- NULL = system job
    reason     VARCHAR(255) NULL,
    changes    JSON NULL,                                 -- {"field": [old, new]}
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    KEY idx_audit_target (table_name, record_id),
    KEY idx_audit_user (changed_by),
    CONSTRAINT fk_audit_user FOREIGN KEY (changed_by) REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Design decisions to be able to explain

- `users` holds login identity only; `customers` holds the business profile, so staff can create a customer before the customer registers.
- **One account per email, case-insensitively, enforced by MySQL:** the `email` column uses the case-insensitive collation `utf8mb4_0900_ai_ci`, so `UNIQUE` treats `Cust@Shop.test` and `cust@shop.TEST` as the same (tested). The service still trims and lower-cases emails before saving.
- Reset tokens are stored as SHA-256 hashes, so a database leak doesn't expose usable links.
- `company_settings` is a single row guaranteed by `CHECK (id = 1)` (tested).
- `number_sequences` gives **gapless** numbers. MySQL has no sequences, and a locked counter row is actually better for invoices: if the transaction rolls back, the increment rolls back too (tested). PostgreSQL sequences, used in our earlier plan, do not roll back.
- `job_queue` is a transactional outbox modelled on Kill Bill's `notifications` table (`effective_date`, `processing_state`, `processing_owner`, `error_count`): a job is inserted **in the same transaction** as the business change, so an email is never sent for a change that rolled back, and a change is never saved without its email.
- `audit_log` follows Kill Bill's `audit_log` and UniBee's `merchant_operation_log`: who changed what, when and why.

---

3. Team-wide conventions you define (write these into the README on day one)

Project layout (GoFrame's standard layout, the same one UniBee's `unibee-api` uses):

```
backend/
  main.go
  manifest/config/config.yaml      -- DB link, SMTP, JWT secret, APP_ENV
  api/{auth,admin,portal}/         -- request/response structs with g.Meta route tags
  internal/
    cmd/  consts/  controller/     -- thin HTTP handlers
    logic/{auth,catalog,pricing,subscription,billing,payment,report,platform}/
    dao/  model/entity/            -- generated by `gf gen dao`
    cronjob/                       -- gcron registrations
  db/migrations/                   -- golang-migrate SQL files
frontend/
  src/{routes.tsx, requests/, stores/, components/, pages/}
docker-compose.yaml                -- mysql, mailhog, api, web
```

API definition style (as in UniBee's `api/` folder):

```go
type CreatePlanReq struct {
    g.Meta `path:"/plans" method:"post" tags:"Plans" summary:"Create plan"`
    Name          string `json:"name" v:"required|length:1,150"`
    BillingPeriod string `json:"billingPeriod" v:"required|in:daily,weekly,monthly,yearly"`
}
```

Use resource routes (`POST /api/plans`, `POST /api/subscriptions/{id}/actions/{action}`), not UniBee's `/xxx/update` style, because the guidelines ask for APIs built around resources.

Success response

```json
{ "data": { ... }, "meta": { "page": 1, "pageSize": 25, "total": 340 } }
```

Error response

```json
{ "error": { "code": "VALIDATION_ERROR", "message": "Please fix the highlighted fields.", "fields": { "email": "Enter a va
```

Code	HTTP	When
`VALIDATION_ERROR`	422	Bad input; `fields` maps field → message
`UNAUTHENTICATED`	401	No or expired session
`FORBIDDEN`	403	Logged in but wrong role
`NOT_FOUND`	404	Missing record **and** another customer's record
`CONFLICT`	409	Duplicate email, invalid status transition, already-billed period
`INTERNAL_ERROR`	500	Unexpected; generic message, full detail only in logs

Your error middleware maps MySQL errors so users never see raw database messages: **1062** (duplicate key) → `CONFLICT`, **3819** (check constraint violated) → `VALIDATION_ERROR`, **1213**/**1205** (deadlock / lock wait timeout) → retry the transaction once, then `CONFLICT`.

Other conventions: money is `DECIMAL(12,2)` in MySQL, `decimal.Decimal` (`github.com/shopspring/decimal`) in Go and a string (`"1560.00"`) in JSON; dates are `YYYY-MM-DD`; timestamps are UTC (`loc=UTC` in the DSN, as UniBee does); lists take `page`, `pageSize` (max 100), `sort` and filters.

Two GoFrame traps (verified in GoFrame v2.6.4 source):

1. `gf gen dao` maps `DECIMAL` columns to **`float64` by default**. Configure `typeMapping` so money becomes `decimal.Decimal`:

```yaml
```

```yaml
gfcli:
  gen:
    dao:
      - link: "mysql:root:secret@tcp(127.0.0.1:3306)/brightloop"
        typeMapping:
          decimal:
            type:   decimal.Decimal
            import: github.com/shopspring/decimal
```

Add a Phase 1 test that reads a `DECIMAL` column through the DAO and asserts the exact string (`"1180.00"`). If it fails, map `decimal` to `string` and convert with `decimal.NewFromString`. Never use `float64` for money. 2. `Model.LockUpdate()` only adds `FOR UPDATE`; there is no `SKIP LOCKED` helper. Write those queries as raw SQL inside `g.DB().Transaction(ctx, func(ctx context.Context, tx gdb.TX) error { ... })`.

---

4. Platform services you provide to the team

Ship stubs in the first hours so B, C and D can call them.

Package / function	Purpose	Used by
`middleware.Auth`, `middleware.RequireRole(roles...)`	JWT (httpOnly cookie, `golang-jwt/jwt/v5`) and role checks	All
`auth.CustomerID(ctx)`	The logged-in customer's id; every portal query **must** filter by it	C, D
`clock.Today(ctx)`	Today's date. In `APP_ENV=development`, returns `company_settings.test_clock_date` when set	C, D
`sequence.Next(ctx, tx, name)`	Next gapless number inside the caller's transaction	C (subscriptions), D (invoices)
`jobs.Enqueue(ctx, tx, queue, dedupeKey, payload, effectiveAt)`	Insert a job in the caller's transaction	D (emails, reminders), A (reset emails)
`audit.Log(ctx, tx, table, id, action, reason, changes)`	Audit entry in the caller's transaction	All
`settings.Get(ctx)`	Company settings	C, D

`sequence.Next` implementation:

```sql
INSERT INTO number_sequences (name, next_value) VALUES (?, 1)
    ON DUPLICATE KEY UPDATE name = name;              -- create the counter on first use
SELECT next_value FROM number_sequences WHERE name = ? FOR UPDATE;
UPDATE number_sequences SET next_value = next_value + 1 WHERE name = ?;
```

Format as `INV-2026-00001` (prefix from settings, counter name per year).

Test clock (Kill Bill's `ClockMock` + `/test/clock` endpoint; UniBee's per-subscription test clock):

- `PUT /api/dev/clock { "date": "2026-04-01" }`, admin only, **registered only when `APP_ENV=development`**.
- **Forward-only.** UniBee rejects moving a test clock backwards, and Kill Bill's `TestResource` warns that moving the clock back and forth on a running system may cause weird side effects.
- Every date decision in the code goes through `clock.Today(ctx)`. Calling `time.Now()` directly in business logic is a review blocker.

Job worker (`gcron.AddSingleton(ctx, "@every 10s", ...)`, the same cadence UniBee uses for its billing job):

1. In a transaction: `SELECT id ... FROM job_queue WHERE state = 'available' AND effective_at <= ? ORDER BY id LIMIT 20 FOR UPDATE SKIP LOCKED`, then set those rows to `processing` with `locked_by` and `locked_until`.
2. Outside the transaction, run each job (send the email through SMTP).
3. On success: `state = 'done'`, `processed_at`. On failure: `attempts + 1`, `last_error`, `effective_at = now + 2^attempts minutes`, back to `available`; after `max_attempts`, `failed`.
4. Recover stuck jobs: `processing` rows whose `locked_until` has passed go back to `available`.

`AddSingleton` only prevents overlap inside one process; `SKIP LOCKED` makes it safe across processes too (verified: a second session skipped the locked row and returned immediately).

5. API endpoints you own

Method	Route	Access	Notes
POST	`/api/auth/register`	Public	Creates `users` (customer) + `customers` row in **one transaction**
POST	`/api/auth/login`	Public	Same error for wrong email or wrong password
POST	`/api/auth/logout`	Logged in	Clears cookie
GET	`/api/auth/me`	Logged in	id, email, role, customerId
POST	`/api/auth/forgot-password`	Public	**Always 200**; if the account exists, enqueues the reset email via `job_queue`
POST	`/api/auth/reset-password`	Public	Token + new password; marks token used; invalidates other tokens
POST	`/api/staff`	Admin	User (staff) + staff profile in one transaction
GET	`/api/users`	Admin	Paginated, filter by role/active
PATCH	`/api/users/{id}/status`	Admin	Activate/deactivate (not yourself)
GET/POST	`/api/customers`	Admin, Staff	Search by name/email
GET/PATCH	`/api/customers/{id}`	Admin, Staff	
GET/PATCH	`/api/portal/profile`	Customer	Own profile only
GET/PUT	`/api/settings`	Read: admin, staff; write: admin	Company settings
GET	`/api/audit-log`	Admin	Filter by table, record, user, date
PUT	`/api/dev/clock`	Admin, development only	Forward-only test clock

Password rule — must be identical on server and client. The problem says *more than* eight characters, so the minimum is 9.

- **Go's `regexp` (RE2) has no lookaheads:** `^(?=.*[a-z])...` fails to compile (tested: `invalid or unsupported Perl syntax: (?=`). Check each rule separately.
- **Don't use GoFrame's built-in `password3` rule:** its source shows 6–18 characters and a required digit, which is not the problem's rule.
- **bcrypt rejects passwords over 72 bytes** (`golang.org/x/crypto/bcrypt` returns `ErrPasswordTooLong`), so 72 bytes is the maximum.

```go
var (
    hasUpper   = regexp.MustCompile(`[A-Z]`)
    hasLower   = regexp.MustCompile(`[a-z]`)
    hasSpecial = regexp.MustCompile(`[^A-Za-z0-9]`)
)

func PasswordProblems(p string) map[string]string {
    probs := map[string]string{}
    if utf8.RuneCountInString(p) < 9 { probs["length"] = "Use more than 8 characters." }
    if len(p) > 72                   { probs["maxLength"] = "Use at most 72 bytes." }
    if !hasUpper.MatchString(p)      { probs["upper"] = "Add an uppercase letter." }
    if !hasLower.MatchString(p)      { probs["lower"] = "Add a lowercase letter." }
    if !hasSpecial.MatchString(p)    { probs["special"] = "Add a special character." }
    return probs
}
```

In TypeScript use the same three character classes, count characters with `[...p].length` (code points, matching `utf8.RuneCountInString`) and bytes with `new TextEncoder().encode(p).length`.

Reset flow: 32 random bytes (`crypto/rand`) → email the raw token in a link (queued) → store SHA-256 hex with a 30-minute expiry → on reset, hash the incoming token, check unexpired and unused, update the password, set `used_at`.

Email: SMTP from Go's standard `net/smtp` to **MailHog** in Docker Compose. UniBee uses the SendGrid API; we don't, because the guidelines discourage third-party APIs and MailHog runs locally.

6. Frontend you own

Ant Design 5 for all components (as UniBee's portals), Zustand for session state, axios with one interceptor that turns the error format into field errors and `message.error` toasts, React Router 6 with role guards.

- **Auth screens:** login, register (live password checklist), forgot and reset password.
- **App shell:** AntD `Layout` with role-based `Menu`; admin sees configuration, staff sees operations, customers see the portal only.
- **Admin:** staff list and "Add staff" form; activate/deactivate with `Popconfirm`; **Settings** page; **audit log** table; in development, a **test clock** card on the settings page.
- **Customers (staff/admin):** list with search and pagination, detail page with tabs C and D fill in.
- **Portal shell:** "My Subscriptions" (C) and "My Invoices" (D).
- **Shared kit:** money and date formatters (`dayjs`), `StatusTag`, empty and error states, `useIdempotencyKey()` hook (`crypto.randomUUID()` when a form opens) used by D's payment dialog.

Agree theme tokens in the kickoff and set them once in AntD's `ConfigProvider`.

---

7. What you depend on from others

From	What	When
C	Subscriptions tab for customer detail and portal	Phase 3
D	Invoices tab for customer detail and portal	Phase 3

You are mostly upstream; others depend on you.

---

8. Phase-by-phase checklist

Phase 0 — Kickoff (with team)

- [ ] Repo, protected `main`, PR template, branch rules
- [ ] Walk the team through `000001_init.up.sql` and `test_constraints.sh`
- [ ] Agree response format, error mapping, theme

Phase 1 — Foundation

- [ ] `docker-compose.yaml`: `mysql:8.0`, `mailhog`, API, web; `.env.example`
- [ ] golang-migrate wired (`migrate -path db/migrations -database "mysql://...?multiStatements=true" up`); `test_constraints.sh` passes on every laptop
- [ ] `gf gen dao` with the decimal `typeMapping`, and the decimal round-trip test
- [ ] Seed: admin, staff, customer, company settings row, payment terms, close reasons
- [ ] Register/login/logout/me with the password rule and bcrypt
- [ ] Middleware, error mapping, `clock`, `sequence`, `jobs`, `audit` stubs merged
- [ ] **Checkpoint 1:** clean DB → migrate → seed → login with all three roles on every machine

Phase 2 — Core workflows

- [ ] Forgot/reset password via job queue + MailHog
- [ ] Job worker with retries and stuck-job recovery
- [ ] Staff creation, app shell, role menus, route guards, shared kit
- [ ] Customers; settings page; test clock (development only)

Phase 3 — Integration

- [ ] Portal shell with C and D views
- [ ] Review every portal endpoint for the `auth.CustomerID` filter
- [ ] Audit log screen; merge coordination

Phase 4 — Testing and hardening

- [ ] Tests in section 9; clean-machine setup test from the README

Phase 5 — Demo prep

- [ ] README: setup, env vars, migrate, seed, run, test, architecture diagram

---

9. Tests and edge cases you must cover

- Duplicate email in a different case → "already registered" (DB test passes; also test the API message)

- Password with 8 characters; missing uppercase, lowercase or special; 73 bytes → each rejected with its own message, identically on client and server
- Login with wrong password and unknown email → identical message; deactivated user can't log in
- Reset token expired, reused or random → rejected; forgot-password for unknown email → still 200
- Staff calling `POST /api/staff` → 403; customer calling admin/staff endpoints → 403
- Customer A requesting customer B's subscription or invoice → 404 (test on C's and D's endpoints too)
- Rolled-back transaction → no job queued, no number consumed
- Email job fails (stop MailHog) → retried with backoff, then `failed`; the business change is not affected
- Test clock: moving backwards rejected; endpoint absent when `APP_ENV` is not `development`

---

10. Git branches (suggested)

`feature/repo-setup`, `feature/migrations`, `feature/auth-core`, `feature/auth-password-reset`, `feature/platform-jobs-clock-audit`, `feature/staff-management`, `feature/customers-settings`, `feature/app-shell-ui-kit`, `feature/portal-shell`

---

11. Your demo segment

1. Register with a weak password → rule feedback; register successfully.
2. Forgot password → the email appears in MailHog → reset works.
3. As admin, create a staff member; log in as staff and show the restricted menu.
4. Show the audit log entry for the staff creation.
5. Log in as the customer and show the portal only exposes their own data.

---

12. Questions you should be ready to answer

- How is "one account per email" guaranteed, including different letter cases and concurrent requests?
- Why can't the password rule be one Go regex, and why not GoFrame's `password3` rule?
- Why does an email go through `job_queue` instead of being sent directly?
- Why are invoice numbers gapless with a counter row but not with a PostgreSQL sequence?
- Why no Redis, when UniBee uses it?
- How does the whole system fit together? (Be able to explain B, C and D at a high level.)

Member B — Catalog & Pricing

Project: Brightloop Subscription Management System **Stack:** Go 1.22 + GoFrame v2.6.4 · MySQL 8.0 · React 18 + TypeScript + Vite + Ant Design 5
Modules owned: Products, attributes and variants, plans, tax rules, discount rules, and the shared **pricing service**

---

1. Your mission

You own everything the admin configures, and the single source of truth for **how money is calculated**. C uses your pricing service to price quotations and subscription lines; D uses it to build every invoice item: the base items, the discount items and the tax items. If pricing lives in one Go package, the numbers on a quotation, a preview and an invoice always agree.

---

2. Database tables you own

Copied from `db/migrations/000001_init.up.sql` (applied to MySQL 8.0.46; all 62 checks in `db/test_constraints.sh` pass).

```sql
```

```sql
CREATE TABLE products (
    id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(150) NOT NULL,
    product_type VARCHAR(50) NOT NULL,                    -- 'software', 'addon', 'service'
    sales_price  DECIMAL(12,2) NOT NULL,
    cost_price   DECIMAL(12,2) NOT NULL,
    notes        TEXT NULL,
    is_recurring BOOLEAN NOT NULL DEFAULT TRUE,           -- FALSE = one-time item, billed on the first invoice only
    is_active    BOOLEAN NOT NULL DEFAULT TRUE,           -- archive, never delete
    created_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    KEY idx_products_active_name (is_active, name),
    CONSTRAINT chk_products_sales_price CHECK (sales_price >= 0),
    CONSTRAINT chk_products_cost_price  CHECK (cost_price >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE product_attributes (
    id   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(80) NOT NULL,                            -- 'Brand'
    PRIMARY KEY (id),
    UNIQUE KEY uq_attribute_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE attribute_values (
    id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    attribute_id BIGINT UNSIGNED NOT NULL,
    value        VARCHAR(80) NOT NULL,                    -- 'Odoo'
    PRIMARY KEY (id),
    UNIQUE KEY uq_attribute_value (attribute_id, value),
    CONSTRAINT fk_attr_values_attribute FOREIGN KEY (attribute_id) REFERENCES product_attributes (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- price_extra per product × value (as Odoo's product.template.attribute.value)
CREATE TABLE product_attribute_values (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    product_id        BIGINT UNSIGNED NOT NULL,
    attribute_value_id BIGINT UNSIGNED NOT NULL,
    price_extra       DECIMAL(12,2) NOT NULL DEFAULT 0,  -- Brand = Odoo → 560.00 (may be negative; final price checked ≥
    is_active         BOOLEAN NOT NULL DEFAULT TRUE,     -- archived, not deleted, once used
    PRIMARY KEY (id),
    UNIQUE KEY uq_product_attr_value (product_id, attribute_value_id),
    CONSTRAINT fk_pav_product FOREIGN KEY (product_id)         REFERENCES products (id) ON DELETE CASCADE,
    CONSTRAINT fk_pav_value   FOREIGN KEY (attribute_value_id) REFERENCES attribute_values (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE product_variants (
    id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    product_id BIGINT UNSIGNED NOT NULL,
    name       VARCHAR(150) NOT NULL,                     -- 'CRM Suite (Odoo)'
    is_active  BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (id),
    KEY idx_variants_product (product_id),
    CONSTRAINT fk_variants_product FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE product_variant_values (
    variant_id                BIGINT UNSIGNED NOT NULL,
    product_attribute_value_id BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (variant_id, product_attribute_value_id),
    CONSTRAINT fk_pvv_variant FOREIGN KEY (variant_id)                REFERENCES product_variants (id)         ON DELETE
    CONSTRAINT fk_pvv_pav     FOREIGN KEY (product_attribute_value_id) REFERENCES product_attribute_values (id) ON DELETE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```sql
CREATE TABLE plans (
    id                    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name                  VARCHAR(150) NOT NULL,
    price                 DECIMAL(12,2) NOT NULL,
    billing_period        ENUM('daily', 'weekly', 'monthly', 'yearly') NOT NULL,
    billing_interval_count SMALLINT UNSIGNED NOT NULL DEFAULT 1,  -- e.g. 3 × monthly = quarterly (UniBee interval_count)
    min_quantity          INT UNSIGNED NOT NULL DEFAULT 1,
    available_from        DATE NOT NULL,
    available_until       DATE NULL,
    auto_close            BOOLEAN NOT NULL DEFAULT FALSE,
    can_close_early       BOOLEAN NOT NULL DEFAULT TRUE,
    can_pause             BOOLEAN NOT NULL DEFAULT FALSE,
    can_renew             BOOLEAN NOT NULL DEFAULT TRUE,
    is_active             BOOLEAN NOT NULL DEFAULT TRUE,
    created_at            DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at            DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    CONSTRAINT chk_plans_price          CHECK (price >= 0),
    CONSTRAINT chk_plans_interval_count CHECK (billing_interval_count >= 1),
    CONSTRAINT chk_plans_min_quantity   CHECK (min_quantity >= 1),
    CONSTRAINT chk_plans_availability   CHECK (available_until IS NULL OR available_until >= available_from)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE tax_rules (
    id             BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name           VARCHAR(80) NOT NULL,                -- 'GST 18%'
    rate_percent   DECIMAL(5,2) NOT NULL,
    is_default     BOOLEAN NOT NULL DEFAULT FALSE,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE,
    default_marker TINYINT UNSIGNED GENERATED ALWAYS AS (IF(is_default, 1, NULL)) STORED, -- MySQL has no partial indexes
    PRIMARY KEY (id),
    UNIQUE KEY uq_tax_name (name),
    UNIQUE KEY uq_tax_single_default (default_marker),
    CONSTRAINT chk_tax_rate CHECK (rate_percent >= 0 AND rate_percent <= 100)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE product_tax_rules (
    product_id  BIGINT UNSIGNED NOT NULL,
    tax_rule_id BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (product_id, tax_rule_id),
    CONSTRAINT fk_ptr_product FOREIGN KEY (product_id)  REFERENCES products (id) ON DELETE CASCADE,
    CONSTRAINT fk_ptr_tax     FOREIGN KEY (tax_rule_id) REFERENCES tax_rules (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE discount_rules (
    id                        BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name                      VARCHAR(120) NOT NULL,
    discount_type             ENUM('fixed', 'percentage') NOT NULL,
    value                     DECIMAL(12,2) NOT NULL,
    scope                     ENUM('subscription', 'products') NOT NULL,
    customer_id               BIGINT UNSIGNED NULL,      -- NULL = any customer
    min_purchase_amount       DECIMAL(12,2) NULL,
    min_quantity              INT UNSIGNED NULL,
    starts_on                 DATE NOT NULL,
    ends_on                   DATE NOT NULL,
    max_uses                  INT UNSIGNED NULL,         -- total uses; NULL = unlimited
    max_uses_per_customer     INT UNSIGNED NULL,         -- UniBee user_limit
    max_cycles_per_subscription INT UNSIGNED NULL,       -- UniBee cycle_limit: first N billing cycles only
    max_discount_amount       DECIMAL(12,2) NULL,        -- cap per invoice (Odoo discount_max_amount)
    times_used                INT UNSIGNED NOT NULL DEFAULT 0,  -- incremented/decremented by D in the billing transact
    is_active                 BOOLEAN NOT NULL DEFAULT TRUE,
    created_by                BIGINT UNSIGNED NOT NULL,
    created_at                DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
```

```
    KEY idx_discounts_active_dates (is_active, starts_on, ends_on),
    CONSTRAINT fk_discount_customer   FOREIGN KEY (customer_id) REFERENCES customers (id),
    CONSTRAINT fk_discount_created_by FOREIGN KEY (created_by)  REFERENCES users (id),
    CONSTRAINT chk_discount_value      CHECK (value > 0),
    CONSTRAINT chk_discount_percentage CHECK (discount_type <> 'percentage' OR value <= 100),
    CONSTRAINT chk_discount_dates      CHECK (ends_on >= starts_on),
    CONSTRAINT chk_discount_min_amount CHECK (min_purchase_amount IS NULL OR min_purchase_amount >= 0),
    CONSTRAINT chk_discount_min_qty    CHECK (min_quantity IS NULL OR min_quantity >= 1),
    CONSTRAINT chk_discount_max_uses   CHECK (max_uses IS NULL OR max_uses >= 1),
    CONSTRAINT chk_discount_usage      CHECK (max_uses IS NULL OR times_used <= max_uses),
    CONSTRAINT chk_discount_cap        CHECK (max_discount_amount IS NULL OR max_discount_amount > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE discount_rule_products (
    discount_rule_id BIGINT UNSIGNED NOT NULL,
    product_id       BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (discount_rule_id, product_id),
    CONSTRAINT fk_drp_rule    FOREIGN KEY (discount_rule_id) REFERENCES discount_rules (id) ON DELETE CASCADE,
    CONSTRAINT fk_drp_product FOREIGN KEY (product_id)       REFERENCES products (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Design decisions to be able to explain

- Variants are built from **generic attributes**, so "Odoo adds 560" is a row, not hard-coded logic.
- `price_extra` lives on the **product × attribute value** pair, where Odoo keeps it (`product.template.attribute.value.price_extra`). A variant's price is always `sales_price + SUM(price_extra)` of its values, never typed in.
- `is_recurring = FALSE` marks one-time items (a setup fee), billed on the first invoice only.
- `billing_interval_count` lets one plan bill every N periods (3 × monthly = quarterly), as UniBee's `interval_count` does. It defaults to 1, so the problem's daily/weekly/monthly/yearly work unchanged.
- **Only one default tax rule:** MySQL has no partial indexes, so `default_marker` is a generated column that is 1 for the default rule and NULL otherwise; a `UNIQUE` index allows many NULLs but only one 1 (tested).
- `times_used` is `UNSIGNED` with `CHECK (times_used <= max_uses)`: the database refuses both over-use and a negative count (both tested).
- Discount limits beyond the problem's total `max_uses`, adopted from UniBee's discount codes (`user_limit`, `cycle_limit`): `max_uses_per_customer` and `max_cycles_per_subscription` ("first 3 months at 20% off"). Both optional.
- `max_discount_amount` caps a percentage discount per invoice (Odoo's `discount_max_amount`).
- Products are archived, never deleted, because invoices reference them.

Decision to confirm in the kickoff: the plan price is billed as its own recurring item each period; recurring product lines (seats, add-ons) are billed per period at their unit price; one-time products on the first invoice only.

---

3. The pricing service (your most important deliverable)

A pure Go package (`internal/logic/pricing`) with no HTTP code, tested with `testing` + `testify`. All money is `decimal.Decimal`.

Function	Returns	Used by
`UnitPrice(ctx, productID, variantID)`	`sales_price + SUM(price_extra)`; error if below 0	C
`TaxRulesFor(ctx, productID)`	Active rules assigned to the product, else the default rule	C, D
`IsPlanAvailable(plan, date)`	bool + reason	C
`ApplicableDiscounts(ctx, tx, dctx)`	Eligible rules, **rows locked** `FOR UPDATE` in ascending id order	D
`PriceItems(baseItems, discounts, taxRules)`	Base items + **linked** discount items + **linked** tax items, all rounded	C (preview), D (invoice)

The item model (from Kill Bill)

Kill Bill puts every amount on an invoice into one `invoice_items` table with a `type` (`RECURRING`, `FIXED`, `TAX`, `ITEM_ADJ`, …) and a `linked_item_id` that points at the item an adjustment applies to. We use the same idea with four types:

Type	Example	Linked to	Sign
`recurring`	"CRM Suite (Odoo) × 12 seats · 1–31 Mar"	—	⩾ 0
`fixed`	"Setup fee" (one-time product)	—	⩾ 0
`discount`	"Seat volume 10%"	the item it reduces	< 0
`tax`	"GST 18%"	the item it taxes	⩾ 0

The database enforces these rules with `CHECK` constraints (all tested): discount and tax items must be linked, other items must not; only discount items may be negative; a tax item must carry a tax rule.

Calculation rules (write them into the README)

1. **Base items:** `amount = round2(quantity × unit_price)`.
2. **Discounts:** a percentage discount gives each in-scope base item its own discount item, `-round2(item × rate)`. A fixed discount is split across in-scope items in proportion to their amounts, with the rounding remainder on the last item. `max_discount_amount` caps the total the same way. A discount never exceeds the item it reduces.
3. **Taxes:** for each base item and each of its tax rules, a tax item `round2((item + its discount items) × rate)`. Tax is therefore always on the discounted amount, item by item.
4. **Totals:** `subtotal` = Σ base items; `discount_total` = −Σ discount items; `tax_total` = Σ tax items; `total` = Σ all items.
5. **Rounding:** `decimal.Round(2)`. Its source documents `5.45 → 5.5`, i.e. half away from zero; we only round positive amounts (discounts are computed positive, then negated), so this equals half-up.
6. **Stacking:** apply the single best eligible discount per invoice (agree in kickoff).
7. **Display:** the printed invoice groups discount items by rule and tax items by rate ("GST 18% on 4,500.00: 810.00").

Discount eligibility (`ApplicableDiscounts`)

A rule applies only if all are true:

1. `is_active` and `starts_on <= today <= ends_on` (today from A's `clock.Today`)
2. `max_uses IS NULL OR times_used < max_uses`
3. `customer_id IS NULL OR customer_id = invoice customer`
4. `max_uses_per_customer` : count of this customer's non-reversed `discount_applications` for the rule is below it
5. `max_cycles_per_subscription` : count of this subscription's non-reversed applications is below it
6. `min_purchase_amount` / `min_quantity` : met by the in-scope base items
7. Scope `products` : only items whose product is in `discount_rule_products` ; scope `subscription` : all base items

Lock eligible rule rows **in ascending id order** inside D's transaction, so two billing workers can't deadlock and can't both take the last use.

---

4. API endpoints you own

Writes: admin only. Reads: admin and staff.

Method	Route	Notes
GET/POST	`/api/products`	Search, type, active; paginated
GET/PATCH	`/api/products/{id}`	With values, variants, taxes
POST	`/api/products/{id}/archive`	Soft delete
GET/POST	`/api/attributes`	With values
POST	`/api/attributes/{id}/values`	
PUT	`/api/products/{id}/attribute-values`	Offered values with `price_extra`; removing a used value archives it
GET/POST	`/api/products/{id}/variants`	Created from chosen values; price computed
GET/POST	`/api/plans`	Filter: active, available on a date
GET/PATCH	`/api/plans/{id}`	Lifecycle flags, interval count
GET/POST	`/api/tax-rules`	Only one default (DB-enforced)
PATCH	`/api/tax-rules/{id}`	
PUT	`/api/products/{id}/tax-rules`	
GET/POST	`/api/discount-rules`	Filter: active, scheduled, expired, used up
GET/PATCH	`/api/discount-rules/{id}`	Can't lower `max_uses` below `times_used` (DB-enforced)
POST	`/api/pricing/preview`	Lines in → items, subtotal, discounts, taxes, total out

Every write calls A's `audit.Log` in the same transaction.

---

5. Frontend you own

Ant Design 5 tables, forms and tags throughout.

- **Products:** list (search, type, archived toggle); form with sales and cost price, notes, "Recurring" switch; margin shown.
- **Attributes & variants:** attribute admin; on the product page, offered values with price extras, then variants built from them with the computed price shown.
- **Plans:** form with period, "every N" count, minimum quantity, availability dates and the four lifecycle switches with helper text.
- **Taxes:** list and form; mark default; assign on the product form.
- **Discounts:** list with status tags (*Active*, *Scheduled*, *Expired*, *Used up*) and a usage `Progress` bar; form that adapts to type and scope, with the optional limits under an "Advanced" section.

---

6. What you depend on from others

From	What	When
A	Middleware, error mapping, `clock`, `audit`, UI kit; `customers` table	Phase 1
D	Calls `ApplicableDiscounts` and `PriceItems` inside its billing transaction, then writes `discount_applications` and increments `times_used`	Phase 2

---

7. Phase-by-phase checklist

Phase 0 — Kickoff (with team)

- [ ] Walk through your tables in the schema; agree plan-price rule, stacking, rounding

Phase 1 — Foundation

- [ ] Catalog and plan CRUD APIs with validation
- [ ] **Pricing service stub** with the agreed signatures, so C and D aren't blocked

Phase 2 — Core workflows

- [ ] Full pricing service with unit tests (section 8)
- [ ] Taxes, discounts, all admin screens; `/api/pricing/preview`
- [ ] **Checkpoint 2 (team):** a subscription produces a correct invoice with linked discount and tax items

Phase 3 — Integration

- [ ] Pair with D on locking order and limits inside the billing transaction

Phase 4 — Testing and hardening

- [ ] Tests in section 8

Phase 5 — Demo prep

- [ ] Seed: products, Brand attribute with Odoo (+560), monthly/yearly/quarterly plans, two tax rules, active and expired discounts

---

8. Tests and edge cases you must cover

- Odoo variant → unit price = sales price + 560; same "Odoo" value with a different extra on another product
- Product with two taxes → two tax items per base item, each rounded
- 3 × 33.33 with 18% tax → items and totals correct to the cent
- Percentage discount over items with different tax rates → each item's tax computed on its own discounted amount
- Fixed discount split across three items → parts sum exactly to the discount
- Fixed discount larger than the subtotal → capped; `max_discount_amount` caps a percentage discount
- Discount expired, used up, for another customer, below minimums, over per-customer or per-subscription limits → not applied
- Database rejections (already in `test_constraints.sh`): percentage over 100, end before start, two default taxes, `times_used` over `max_uses` or below zero
- Staff creating a discount → 403

---

9. Git branches (suggested)

`feature/catalog-products`, `feature/catalog-variants`, `feature/plans`, `feature/tax-rules`, `feature/discount-rules`, `feature/pricing-service`

---

10. Your demo segment

1. As admin, create a product, add Brand = Odoo (+560), and show the variant price computed automatically.
2. Create a monthly plan and a quarterly plan (every 3 months); explain the lifecycle switches.
3. Create a tax rule and a 10% discount with a minimum of 10 seats, 5 total uses and a 3-cycle limit.
4. Open the pricing preview: base item, its linked discount item, its linked tax item.
5. Log in as staff and show discount creation is blocked.

---

11. Questions you should be ready to answer

- Why do discount and tax items link to the item they apply to (Kill Bill's `linked_item_id`)?
- Why does `price_extra` sit on the product × value pair?
- How does MySQL guarantee a single default tax rule without partial indexes?
- How does the database stop a discount being over-used, even under concurrent billing?
- Why lock discount rows in ascending id order?
- How does the full flow work from signup to dashboard? (Explain A, C and D at a high level.)

Member C — Sales & Subscriptions

Project: Brightloop Subscription Management System **Stack:** Go 1.22 + GoFrame v2.6.4 · MySQL 8.0 · React 18 + TypeScript + Vite + Ant Design 5
Modules owned: Quotation templates, quotations, subscriptions and lines, the **lifecycle state machine**, **billing-period date logic**, renewals, pauses, closures, add-ons, MRR figures, and the portal's "My Subscriptions"

---

1. Your mission

You turn an offer into a running subscription and keep its status and dates correct. D's billing engine trusts your `next_billing_date`, your period calculation and your close dates completely, so date logic is the most important code you write. Every status change follows one state machine, is recorded in history, and respects the plan's settings.

---

2. Database tables you own

Copied from `db/migrations/000001_init.up.sql` (applied to MySQL 8.0.46; all 62 checks in `db/test_constraints.sh` pass).

```sql
```

```sql
CREATE TABLE payment_terms (
    id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name     VARCHAR(60) NOT NULL,                      -- 'Immediate', 'Net 15', 'Net 30'
    due_days INT UNSIGNED NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_payment_terms_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE close_reasons (
    id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name     VARCHAR(80) NOT NULL,                      -- 'Too expensive', 'Renewed', 'Ended', 'Quotation expired'
    is_churn BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (id),
    UNIQUE KEY uq_close_reasons_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE quotation_templates (
    id               BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name             VARCHAR(150) NOT NULL,
    plan_id          BIGINT UNSIGNED NOT NULL,
    validity_days    INT UNSIGNED NOT NULL,
    payment_terms_id BIGINT UNSIGNED NULL,
    terms            TEXT NULL,
    notes            TEXT NULL,
    is_active        BOOLEAN NOT NULL DEFAULT TRUE,
    created_by       BIGINT UNSIGNED NOT NULL,
    created_at       DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    CONSTRAINT fk_templates_plan       FOREIGN KEY (plan_id)          REFERENCES plans (id),
    CONSTRAINT fk_templates_terms      FOREIGN KEY (payment_terms_id) REFERENCES payment_terms (id),
    CONSTRAINT fk_templates_created_by FOREIGN KEY (created_by)       REFERENCES users (id),
    CONSTRAINT chk_templates_validity  CHECK (validity_days > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE quotation_template_lines (
    id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    template_id BIGINT UNSIGNED NOT NULL,
    product_id  BIGINT UNSIGNED NOT NULL,
    variant_id  BIGINT UNSIGNED NULL,
    quantity    INT UNSIGNED NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_tlines_template FOREIGN KEY (template_id) REFERENCES quotation_templates (id) ON DELETE CASCADE,
    CONSTRAINT fk_tlines_product  FOREIGN KEY (product_id)  REFERENCES products (id),
    CONSTRAINT fk_tlines_variant  FOREIGN KEY (variant_id)  REFERENCES product_variants (id),
    CONSTRAINT chk_tlines_quantity CHECK (quantity > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE subscriptions (
    id                   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    number               VARCHAR(30) NOT NULL,          -- 'SUB-2026-00042' from number_sequences
    customer_id          BIGINT UNSIGNED NOT NULL,
    plan_id              BIGINT UNSIGNED NOT NULL,
    template_id          BIGINT UNSIGNED NULL,
    payment_terms_id     BIGINT UNSIGNED NOT NULL,
    status               ENUM('draft', 'quotation', 'confirmed', 'active', 'paused', 'closed') NOT NULL DEFAULT 'draft',
    start_date           DATE NOT NULL,
    expiry_date          DATE NULL,                     -- NULL = open-ended
    billing_anchor_date  DATE NULL,                     -- periods are computed from here (Kill Bill BCD / UniBee billi
    next_billing_date    DATE NULL,                     -- read by D's billing engine
    quotation_valid_until DATE NULL,
    scheduled_close_date DATE NULL,                     -- close at end of term (Kill Bill END_OF_TERM / UniBee cancel_a
    recurring_total      DECIMAL(12,2) NOT NULL DEFAULT 0, -- per billing period, before tax and discounts
    recurring_monthly    DECIMAL(12,2) NOT NULL DEFAULT 0, -- normalised to one month → MRR
    renewed_from_id      BIGINT UNSIGNED NULL,
```

```
    terms                TEXT NULL,
    salesperson_id       BIGINT UNSIGNED NULL,
    paused_at            DATETIME(3) NULL,
    closed_at            DATETIME(3) NULL,
    close_reason_id      BIGINT UNSIGNED NULL,
    close_note           VARCHAR(200) NULL,
    created_at           DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at           DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    open_renewal_marker  BIGINT UNSIGNED GENERATED ALWAYS AS
                         (IF(renewed_from_id IS NOT NULL AND status <> 'closed', renewed_from_id, NULL)) STORED,
    PRIMARY KEY (id),
    UNIQUE KEY uq_subscriptions_number (number),
    UNIQUE KEY uq_subscriptions_one_open_renewal (open_renewal_marker),
    KEY idx_subs_status_next_billing (status, next_billing_date),
    KEY idx_subs_status_expiry (status, expiry_date),
    KEY idx_subs_status_scheduled_close (status, scheduled_close_date),
    CONSTRAINT fk_subs_customer     FOREIGN KEY (customer_id)      REFERENCES customers (id),
    CONSTRAINT fk_subs_plan         FOREIGN KEY (plan_id)          REFERENCES plans (id),
    CONSTRAINT fk_subs_template     FOREIGN KEY (template_id)      REFERENCES quotation_templates (id),
    CONSTRAINT fk_subs_terms        FOREIGN KEY (payment_terms_id) REFERENCES payment_terms (id),
    CONSTRAINT fk_subs_renewed_from FOREIGN KEY (renewed_from_id)  REFERENCES subscriptions (id),
    CONSTRAINT fk_subs_salesperson  FOREIGN KEY (salesperson_id)   REFERENCES users (id),
    CONSTRAINT fk_subs_close_reason FOREIGN KEY (close_reason_id)  REFERENCES close_reasons (id),
    CONSTRAINT chk_subs_expiry      CHECK (expiry_date IS NULL OR expiry_date >= start_date),
    CONSTRAINT chk_subs_recurring   CHECK (recurring_total >= 0 AND recurring_monthly >= 0),
    CONSTRAINT chk_subs_closed      CHECK ((status = 'closed') = (closed_at IS NOT NULL))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE subscription_lines (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    subscription_id BIGINT UNSIGNED NOT NULL,
    product_id      BIGINT UNSIGNED NOT NULL,
    variant_id      BIGINT UNSIGNED NULL,
    quantity        INT UNSIGNED NOT NULL,
    unit_price      DECIMAL(12,2) NOT NULL,              -- agreed price, from B's GetUnitPrice
    amount          DECIMAL(14,2) GENERATED ALWAYS AS (quantity * unit_price) STORED,
    PRIMARY KEY (id),
    KEY idx_sub_lines_subscription (subscription_id),
    CONSTRAINT fk_sub_lines_subscription FOREIGN KEY (subscription_id) REFERENCES subscriptions (id) ON DELETE CASCADE,
    CONSTRAINT fk_sub_lines_product      FOREIGN KEY (product_id)      REFERENCES products (id),
    CONSTRAINT fk_sub_lines_variant      FOREIGN KEY (variant_id)      REFERENCES product_variants (id),
    CONSTRAINT chk_sub_lines_quantity    CHECK (quantity > 0),
    CONSTRAINT chk_sub_lines_price       CHECK (unit_price >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE subscription_status_history (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    subscription_id BIGINT UNSIGNED NOT NULL,
    from_status     ENUM('draft', 'quotation', 'confirmed', 'active', 'paused', 'closed') NULL,
    to_status       ENUM('draft', 'quotation', 'confirmed', 'active', 'paused', 'closed') NOT NULL,
    changed_by      BIGINT UNSIGNED NULL,                -- NULL = scheduled job
    note            VARCHAR(255) NULL,
    changed_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    KEY idx_sub_history (subscription_id, changed_at),
    CONSTRAINT fk_sub_history_subscription FOREIGN KEY (subscription_id) REFERENCES subscriptions (id) ON DELETE CASCADE,
    CONSTRAINT fk_sub_history_user         FOREIGN KEY (changed_by)      REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Design decisions to be able to explain

- A quotation is a subscription in `draft` / `quotation` status, not a separate table: the number, lines and history stay continuous from offer to closure.
- `billing_anchor_date` is the date periods are counted from, like Kill Bill's bill cycle day (`billing_cycle_day_local`) and UniBee's `billing_cycle_anchor`. It is set to `start_date` on activation; periods are always computed from it, so they never drift.

- `scheduled_close_date` implements **close at end of term**: Kill Bill's `END_OF_TERM` cancellation policy and UniBee's `cancel_at_period_end`.
- **At most one open renewal per subscription:** MySQL has no partial unique index, so `open_renewal_marker` is a generated column that holds `renewed_from_id` only while the renewal isn't closed; a `UNIQUE` index on it blocks a second open renewal but allows a new one after the first is closed (both tested).
- `CHECK ((status = 'closed') = (closed_at IS NOT NULL))`: a closed subscription always has a close time, and only closed ones do (tested).
- `subscription_lines.amount` is a generated column (`quantity × unit_price`), so it can never disagree with its inputs (tested: 3 × 33.33 = 99.99).
- `recurring_total` / `recurring_monthly` are recalculated whenever lines change, so D gets **MRR** with one `SUM` (measured: 19 ms over 10,000 subscriptions).
- D's "what's due" query took 9 ms for a batch of 100 among 10,000 seeded subscriptions. At that size MySQL chose the primary key (because of `ORDER BY id`); the `(status, next_billing_date)` index is there for larger tables. Check with `EXPLAIN` in Phase 4.

How our statuses map to the reference systems

Ours	Kill Bill entitlement state	UniBee subscription status
draft, quotation	— (quotes live outside Kill Bill)	0 Init
confirmed (future start)	PENDING	1 Pending
active	ACTIVE	2 Active
paused	BLOCKED (via `blocking_states.block_billing`)	6 Suspend
closed	CANCELLED	4 Cancel / 5 Expire

---

3. The lifecycle state machine

Action	From → To	Guard (`CONFLICT` if false)
`send`	draft → quotation	≥1 line; plan available on start date (B's `IsPlanAvailable`); total quantity ≥ `plan.min_quantity`. Sets `quotation_valid_until` = today + template days (or A's company default)
`accept`	quotation → confirmed	today ≤ `quotation_valid_until`
`activate`	confirmed → active	Sets `billing_anchor_date = start_date`, `next_billing_date = start_date`. Future start dates are activated by the daily job
`pause`	active → paused	`plan.can_pause`; sets `paused_at`
`resume`	paused → active	`next_billing_date` = first period start ≥ today (paused periods are never billed)
`renew`	active (unchanged)	`plan.can_renew`, has `expiry_date`, no open renewal. Creates a **new** subscription in `quotation` with `renewed_from_id`, same plan and lines (re-priced by B), `start_date = expiry + 1 day`
`close`	active/paused → closed, or scheduled	Body `{ when: "immediate" | "end_of_period", reasonId, note }`. Immediate before expiry needs `plan.can_close_early`; sets `closed_at`. End of period sets `scheduled_close_date = next_billing_date` and stays active
`undo_close`	active (scheduled)	Clears `scheduled_close_date` before it happens (Kill Bill's `UNCANCEL`)
`cancel`	draft/quotation → closed	Reason required

Implementation rules

- One `subscription.Transition(ctx, id, action, input, user)` holds this table. Nothing else writes `status`.

- Inside `g.DB().Transaction`: `SELECT ... FOR UPDATE` the subscription (GoFrame's `LockUpdate()` is fine here), check the guard, update, insert `subscription_status_history`, call A's `audit.Log`. Two staff clicking "close" at once: the second waits, then sees `closed` and gets `CONFLICT`.
- Lines are editable in `draft` / `quotation`. On an **active** subscription, staff may add or remove add-ons; the change applies from the next period (no proration), recalculates `recurring_*`, and is noted in history. D's invoices are unaffected because they snapshot items.

Daily lifecycle steps (D's nightly scheduler calls them in this order around billing):

1. `lifecycle.ActivateDue(ctx, today)`: `confirmed` with `start_date <= today` → `active`.
2. *(D's billing run)*
3. `lifecycle.CloseDue(ctx, today)`, each with a close reason:
   - `scheduled_close_date <= today` → closed ("Cancelled at end of term")
   - `plan.auto_close` and `expiry_date < today` → closed ("Ended"), unless a renewal of it is confirmed or active
   - an activated renewal → its original closed ("Renewed", not churn)
4. Past expiry with no auto-close: stays active, listed under **"Expired — action needed"**.

---

4. Billing-period date logic (`internal/logic/subscription/billingdate`)

Period *k* starts at `anchor + k × interval`, where the interval is the plan's period × `billing_interval_count`. Period *k* ends the day before period *k+1* starts.

Go trap: `time.AddDate` normalises overflow. Adding one month to 31 January gives 3 March (in 2026), not 28 February. Month and year steps must clamp to the last day of the month:

```go
// addMonthsClamped adds months to t, clamping the day to the target month's last day.
func addMonthsClamped(t time.Time, months int) time.Time {
    y, m, d := t.Date()
    first := time.Date(y, m+time.Month(months), 1, 0, 0, 0, 0, time.UTC)
    last := first.AddDate(0, 1, -1).Day()
    if d > last {
        d = last
    }
    return time.Date(first.Year(), first.Month(), d, 0, 0, 0, 0, time.UTC)
}

// PeriodStart returns the start of billing period k (k = 0 is the anchor).
func PeriodStart(anchor time.Time, period string, every, k int) time.Time {
    switch period {
    case "daily":
        return anchor.AddDate(0, 0, every*k)
    case "weekly":
        return anchor.AddDate(0, 0, 7*every*k)
    case "monthly":
        return addMonthsClamped(anchor, every*k)
    default: // "yearly"
        return addMonthsClamped(anchor, 12*every*k)
    }
}
```

Because every period is computed **from the anchor**, a 31 January subscription bills 31 Jan → 28 Feb → 31 Mar → 30 Apr, with no drift. These cases are checked by `reference/billingdate/billingdate_test.go` (passing on Go 1.22.2); copy the package and its tests into the backend in Phase 1.

Function	Purpose	Used by
`PeriodStart(anchor, period, every, k)` / `PeriodEnd(...)`	Period boundaries	D
`NextPeriodOnOrAfter(sub, date)`	Resume, scheduled close	You, D
`MonthlyEquivalent(amount, period, every)`	`recurring_monthly`: daily × 365/12, weekly × 52/12, monthly × 1, yearly ÷ 12, each ÷ `every`, rounded to 2 dp	You, D

---

5. API endpoints you own

Method	Route	Access	Notes
GET/POST	`/api/quotation-templates`	Read: staff, admin; write: agree in kickoff	
GET/PATCH	`/api/quotation-templates/{id}`	Same	
GET	`/api/payment-terms`, `/api/close-reasons`	Logged in	Seeded
POST	`/api/subscriptions`	Staff, admin	Blank draft or `{ templateId, customerId, startDate }`; number from A's `sequence.Next`
GET	`/api/subscriptions`	Staff, admin	Filters: status, customer, plan, number, dates, "expired — action needed", "closing at end of term"; paginated
GET	`/api/subscriptions/{id}`	Staff, admin	Lines, pricing preview, history, `allowedActions`, renewal links
PATCH	`/api/subscriptions/{id}`	Staff, admin	Header fields in draft/quotation
PUT	`/api/subscriptions/{id}/lines`	Staff, admin	Draft/quotation; add-ons on active (next period)
POST	`/api/subscriptions/{id}/actions/{action}`	Staff, admin	Actions in section 3
GET	`/api/portal/subscriptions[/{id}]`	Customer	Own only (A's `auth.CustomerID`), else 404
POST	`/api/portal/subscriptions/{id}/actions/{action}`	Customer	`accept`; `pause` / `resume` / `close` / `undo_close` when the plan allows

The detail response includes `allowedActions`, so the UI only shows valid buttons; the server re-checks every action.

---

6. Frontend you own

- **Templates:** list and form (plan, validity days, payment terms, terms text, product lines with variant picker).
- **New quotation:** customer → template or blank → lines prefilled and priced → edit quantities → live totals from B's `/api/pricing/preview` → save or send.
- **Subscription list:** AntD `Table` with status tags, quick filters per status, "Expired — action needed" and "Closing at end of term".
- **Subscription detail:** header, lines, recurring amount and monthly equivalent, AntD `Timeline` from history, renewal links, action buttons from `allowedActions`. The close dialog asks for a reason and "Now / At end of period".
- **Customer tab** and **portal "My Subscriptions"** with the customer actions above.

---

7. What you depend on from others

From	What	When
A	Auth, `auth.CustomerID`, `clock.Today`, `sequence.Next`, `audit.Log`, settings, UI kit	Phase 1
B	`UnitPrice`, `IsPlanAvailable`, `/api/pricing/preview`	Stub Phase 1, real Phase 2
D	Calls your date functions and lifecycle steps	Phase 2

---

8. Phase-by-phase checklist

Phase 0 — Kickoff (with team)

- [ ] Walk the team through the state machine and the date rules

Phase 1 — Foundation

- [ ] Template and subscription CRUD APIs
- [ ] `billingdate` package with tests (month-end, leap year, every-N), exported for D

Phase 2 — Core workflows

- [ ] Create from template; `Transition` with guards, locking, history, audit
- [ ] Template, quotation, list and detail screens
- [ ] **Checkpoint 2 (team):** quote → active subscription → D generates a correct invoice

Phase 3 — Integration

- [ ] Lifecycle steps (activate, scheduled close, auto-close, renewal hand-off)
- [ ] Renewals, end-of-term close, add-ons; portal and customer tab

Phase 4 — Testing and hardening

- [ ] Tests in section 9

Phase 5 — Demo prep

- [ ] Seed subscriptions in every status, one expiring soon, one paused, one closing at end of term

---

9. Tests and edge cases you must cover

- Every invalid transition → `CONFLICT` with a clear message; expired quotation can't be accepted
- Quantity below plan minimum; plan unavailable on start date → rejected on send
- Concurrent close → one succeeds, one `CONFLICT`
- Dates: monthly from 31 Jan; yearly from 29 Feb 2024; weekly; daily; every-3-months; never use raw `AddDate` for months
- Future start → activated by the job on that date (move A's test clock to show it)
- Resume after a long pause → next billing date is the first period start on or after today; no back-billing
- Close at end of period → still active and billed for the current period, closed on `scheduled_close_date`; `undo_close` before then keeps it running
- Renewal: linked quotation starting the day after expiry; a second open renewal blocked (DB test); activating it closes the original as "Renewed"
- `recurring_monthly` correct for every period type; one-time products excluded
- Add-on on an active subscription → only the next invoice changes
- Customer A acting on customer B's subscription → 404

---

10. Git branches (suggested)

`feature/quotation-templates`, `feature/subscriptions-crud`, `feature/billing-dates`, `feature/subscription-lifecycle`, `feature/lifecycle-jobs`, `feature/renewals-and-closing`, `feature/portal-subscriptions`

---

11. Your demo segment

1. As staff, create a quotation from a template; lines and totals fill in automatically.
2. Send it; as the customer, accept it in the portal; as staff, activate it and show the timeline.
3. Try an invalid action and show it is refused.
4. Close a subscription "at end of period": it stays active until the date, then closes (use the test clock).
5. Renew a yearly subscription: a linked renewal quotation appears, starting the day after expiry.

---

12. Questions you should be ready to answer

- Why are billing periods computed from an anchor, and why can't you use `time.AddDate` for months?
- How do you prevent two people changing a status at the same moment?
- What's the difference between closing now and closing at the end of the period?
- Why does renewal create a new linked subscription?
- How does MySQL enforce "one open renewal" without partial indexes?
- How does the full flow work from signup to dashboard? (Explain A, B and D at a high level.)

Member D — Billing, Payments & Reporting

Project: Brightloop Subscription Management System **Stack:** Go 1.22 + GoFrame v2.6.4 · MySQL 8.0 · React 18 + TypeScript + Vite + Ant Design 5
Modules owned: Recurring billing engine, invoices and invoice items, invoice actions and PDF, discount applications, payments, overdue reminders, the finance dashboard, portal "My Invoices", seed data and performance testing

---

1. Your mission

On billing day hundreds of subscriptions are due at once. Your engine must create every invoice that is due, **never twice and never skipped**, even when it is re-run, crashes halfway, or two workers run at the same time. Then you track the money: payments, balances, overdue invoices, and the finance dashboard, which must answer in **under two seconds** with thousands of subscriptions.

---

2. Database tables you own

Copied from `db/migrations/000001_init.up.sql` (applied to MySQL 8.0.46; all 62 checks in `db/test_constraints.sh` pass).

sql

```sql
CREATE TABLE invoices (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    number          VARCHAR(30) NULL,                    -- NULL while draft; assigned on confirm
    customer_id     BIGINT UNSIGNED NOT NULL,
    subscription_id BIGINT UNSIGNED NULL,
    status          ENUM('draft', 'confirmed', 'paid', 'cancelled') NOT NULL DEFAULT 'draft',
    invoice_date    DATE NOT NULL,
    due_date        DATE NOT NULL,
    period_start    DATE NULL,
    period_end      DATE NULL,
    billing_name    VARCHAR(150) NOT NULL,               -- customer snapshot
    billing_address TEXT NULL,
    subtotal        DECIMAL(12,2) NOT NULL DEFAULT 0,    -- recurring + fixed items
    discount_total  DECIMAL(12,2) NOT NULL DEFAULT 0,    -- positive number (sum of negative discount items, sign flippe
    tax_total       DECIMAL(12,2) NOT NULL DEFAULT 0,
    total           DECIMAL(12,2) NOT NULL DEFAULT 0,    -- = SUM(invoice_items.amount)
    amount_paid     DECIMAL(12,2) NOT NULL DEFAULT 0,    -- maintained in the payment transaction
    payment_status  VARCHAR(10) GENERATED ALWAYS AS (
                        CASE WHEN amount_paid = 0 THEN 'not_paid'
                             WHEN amount_paid < total THEN 'partial'
                             ELSE 'paid' END) STORED,
    confirmed_at    DATETIME(3) NULL,
    sent_at         DATETIME(3) NULL,
    cancelled_at    DATETIME(3) NULL,
    cancel_reason   VARCHAR(200) NULL,
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_invoices_number (number),              -- multiple NULLs allowed (drafts)
    KEY idx_invoices_status_due (status, due_date),
    KEY idx_invoices_date (invoice_date),
    CONSTRAINT fk_invoices_customer     FOREIGN KEY (customer_id)     REFERENCES customers (id),
    CONSTRAINT fk_invoices_subscription FOREIGN KEY (subscription_id) REFERENCES subscriptions (id),
    CONSTRAINT chk_invoices_total       CHECK (total >= 0),
    CONSTRAINT chk_invoices_paid_range  CHECK (amount_paid >= 0 AND amount_paid <= total),
    CONSTRAINT chk_invoices_due         CHECK (due_date >= invoice_date),
    CONSTRAINT chk_invoices_numbered    CHECK (status IN ('draft', 'cancelled') OR number IS NOT NULL),
    CONSTRAINT chk_invoices_paid_status CHECK (status <> 'paid' OR amount_paid = total),
    CONSTRAINT chk_invoices_period      CHECK (period_end IS NULL OR period_end >= period_start)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- One table for every amount on an invoice (Kill Bill invoice_items pattern).
-- Items are append-only and invoices are never deleted (drafts are cancelled), so no ON DELETE CASCADE here.
CREATE TABLE invoice_items (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    invoice_id      BIGINT UNSIGNED NOT NULL,
    item_type       ENUM('recurring', 'fixed', 'discount', 'tax') NOT NULL,
    linked_item_id  BIGINT UNSIGNED NULL,                -- discount/tax items point at the item they apply to
    product_id      BIGINT UNSIGNED NULL,                -- NULL for the plan line, discounts and taxes
    plan_id         BIGINT UNSIGNED NULL,
    discount_rule_id BIGINT UNSIGNED NULL,
    tax_rule_id     BIGINT UNSIGNED NULL,
    description     VARCHAR(255) NOT NULL,               -- snapshot, e.g. 'CRM Suite (Odoo) · 1 Mar – 31 Mar 2026'
    service_start   DATE NULL,                           -- service period of recurring items
    service_end     DATE NULL,
    quantity        INT UNSIGNED NOT NULL DEFAULT 1,
    unit_price      DECIMAL(12,2) NULL,                  -- snapshot for recurring/fixed items
    rate_percent    DECIMAL(5,2) NULL,                   -- snapshot: tax rate or discount percentage
    amount          DECIMAL(12,2) NOT NULL,              -- negative only for discount items
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    KEY idx_items_invoice (invoice_id),
    KEY idx_items_linked (linked_item_id),
    CONSTRAINT fk_items_invoice  FOREIGN KEY (invoice_id)      REFERENCES invoices (id),   -- invoices are never deleted
```

```
    CONSTRAINT fk_items_linked   FOREIGN KEY (linked_item_id)   REFERENCES invoice_items (id),
    CONSTRAINT fk_items_product  FOREIGN KEY (product_id)       REFERENCES products (id),
    CONSTRAINT fk_items_plan     FOREIGN KEY (plan_id)          REFERENCES plans (id),
    CONSTRAINT fk_items_discount FOREIGN KEY (discount_rule_id) REFERENCES discount_rules (id),
    CONSTRAINT fk_items_tax      FOREIGN KEY (tax_rule_id)      REFERENCES tax_rules (id),
    CONSTRAINT chk_items_link     CHECK ((item_type IN ('discount', 'tax')) = (linked_item_id IS NOT NULL)),
    CONSTRAINT chk_items_discount CHECK ((item_type = 'discount') = (discount_rule_id IS NOT NULL)),
    CONSTRAINT chk_items_tax      CHECK ((item_type = 'tax') = (tax_rule_id IS NOT NULL)),
    CONSTRAINT chk_items_sign     CHECK ((item_type = 'discount' AND amount < 0) OR (item_type <> 'discount' AND amount >=
    CONSTRAINT chk_items_quantity CHECK (quantity > 0),
    CONSTRAINT chk_items_service  CHECK (service_end IS NULL OR service_end >= service_start)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- The duplicate-billing guard: one invoice per subscription per period
CREATE TABLE billing_records (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    subscription_id BIGINT UNSIGNED NOT NULL,
    period_start    DATE NOT NULL,
    period_end      DATE NOT NULL,
    invoice_id      BIGINT UNSIGNED NOT NULL,
    inputs          JSON NULL,                          -- snapshot of what billing used (Kill Bill invoice_billing_ever
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_billing_period (subscription_id, period_start),
    UNIQUE KEY uq_billing_invoice (invoice_id),
    CONSTRAINT fk_billing_subscription FOREIGN KEY (subscription_id) REFERENCES subscriptions (id),
    CONSTRAINT fk_billing_invoice      FOREIGN KEY (invoice_id)      REFERENCES invoices (id),
    CONSTRAINT chk_billing_period      CHECK (period_end >= period_start)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE discount_applications (
    id               BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    discount_rule_id BIGINT UNSIGNED NOT NULL,
    invoice_id       BIGINT UNSIGNED NOT NULL,
    customer_id      BIGINT UNSIGNED NOT NULL,
    subscription_id  BIGINT UNSIGNED NULL,
    amount           DECIMAL(12,2) NOT NULL,            -- positive: total discount granted on this invoice
    reversed_at      DATETIME(3) NULL,                  -- set when the invoice is cancelled (UniBee 'rollback')
    applied_at       DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_discount_once_per_invoice (discount_rule_id, invoice_id),
    KEY idx_da_rule_customer (discount_rule_id, customer_id),
    KEY idx_da_rule_subscription (discount_rule_id, subscription_id),
    CONSTRAINT fk_da_rule         FOREIGN KEY (discount_rule_id) REFERENCES discount_rules (id),
    CONSTRAINT fk_da_invoice      FOREIGN KEY (invoice_id)       REFERENCES invoices (id),
    CONSTRAINT fk_da_customer     FOREIGN KEY (customer_id)      REFERENCES customers (id),
    CONSTRAINT fk_da_subscription FOREIGN KEY (subscription_id)  REFERENCES subscriptions (id),
    CONSTRAINT chk_da_amount      CHECK (amount > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE payments (
    id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    invoice_id      BIGINT UNSIGNED NOT NULL,
    method          ENUM('bank_transfer', 'card', 'cash', 'cheque', 'upi', 'other') NOT NULL,
    amount          DECIMAL(12,2) NOT NULL,
    payment_date    DATE NOT NULL,
    note            VARCHAR(250) NULL,
    idempotency_key CHAR(36) NOT NULL,                  -- UUID generated by the form; a double submit can't record twi
    recorded_by     BIGINT UNSIGNED NOT NULL,
    created_at      DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_payments_idempotency (idempotency_key),
    KEY idx_payments_invoice (invoice_id),
    KEY idx_payments_date (payment_date),
```

```
    CONSTRAINT fk_payments_invoice FOREIGN KEY (invoice_id)  REFERENCES invoices (id),
    CONSTRAINT fk_payments_user    FOREIGN KEY (recorded_by) REFERENCES users (id),
    CONSTRAINT chk_payments_amount CHECK (amount > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE billing_runs (
    id            BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    as_of_date    DATE NOT NULL,
    triggered_by  BIGINT UNSIGNED NULL,                  -- NULL = scheduler
    dry_run       BOOLEAN NOT NULL DEFAULT FALSE,
    started_at    DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    finished_at   DATETIME(3) NULL,
    due_count     INT UNSIGNED NOT NULL DEFAULT 0,
    created_count INT UNSIGNED NOT NULL DEFAULT 0,
    skipped_count INT UNSIGNED NOT NULL DEFAULT 0,
    failed_count  INT UNSIGNED NOT NULL DEFAULT 0,
    errors        JSON NULL,                             -- [{"subscription_id": .., "message": ..}]
    PRIMARY KEY (id),
    KEY idx_billing_runs_started (started_at),
    CONSTRAINT fk_billing_runs_user FOREIGN KEY (triggered_by) REFERENCES users (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Design decisions to be able to explain

- `billing_records UNIQUE (subscription_id, period_start)` is the last line of defence: the database refuses a second invoice for the same period, whatever the code does (tested). Neither reference system has this: UniBee decides in application code from the latest invoice's status under a Redis lock, and Kill Bill recomputes items and writes only the difference.
- `invoice_items` **follows Kill Bill's** `invoice_items` **:** one table for every amount, a `type`, and `linked_item_id` for discounts and taxes. Like Kill Bill's table (which has no `updated_*` columns), items are **append-only**. `CHECK` constraints enforce the linking and sign rules (tested).
- **Invoice numbers are assigned on confirm**, from A's gapless counter. Drafts have `number = NULL`; a `UNIQUE` index allows many NULLs (tested). `CHECK (status IN ('draft','cancelled') OR number IS NOT NULL)` guarantees every confirmed or paid invoice has one (tested).
- `payment_status` is a generated column (`not_paid` / `partial` / `paid`) next to the problem's `status`, as Odoo has `payment_state` and Kill Bill derives balances separately. `CHECK (status <> 'paid' OR amount_paid = total)` means "Paid" can't be set early, and `CHECK (amount_paid <= total)` blocks overpayment (both tested).
- `payments.idempotency_key` (UUID from the form) makes a double submit harmless, like Kill Bill's unique `payments.external_key` and UniBee's `unique_id` columns (tested).
- `billing_records.inputs` keeps a JSON snapshot of what billing used (lines, prices, discounts considered), like Kill Bill's `invoice_billing_events`. JSON is fine here because it is a debugging log, not business data the app queries.
- Invoices are **never deleted** (drafts are cancelled), so item and discount foreign keys have no `ON DELETE CASCADE`. MySQL rejects deleting an invoice that still has items or a billing record (tested: error 1451).

---

3. The billing engine (`internal/logic/billing`)

What we take from each reference

Idea	Source (observed)	Our version
Poll for due subscriptions on a timer	UniBee: `gcron.AddSingleton(ctx, "@every 10s", ...)` selecting subscriptions whose `dunning_time` has passed	Nightly `gcron` job + "Run billing now" button; due = `status = 'active' AND next_billing_date <= today`
Lock while billing one subscription	UniBee: Redis `SET key 1 NX EX 60` per subscription. Kill Bill: global lock per account around invoicing and payment (`ACCNT_INV_PAY`)	MySQL row lock `FOR UPDATE SKIP LOCKED` on the subscription, held exactly as long as the transaction
Compute what *should* exist, write only what's missing	Kill Bill: "proposed items are ALL items since beginning of time", merged with existing items	Expected periods from `next_billing_date` to today, minus periods already in `billing_records`
Safety bound	Kill Bill: sanity check against duplicate items for a period (on by default) and at most **15** items per subscription per day	At most **31** periods per subscription per run; above that, refuse and log for a human
Preview without writing	Kill Bill dry-run invoices; UniBee `PreviewSubscriptionNextInvoice`	`GET /api/subscriptions/{id}/upcoming-invoice` and `dryRun` billing runs

Why not Redis like UniBee: its lock is released with a plain `DEL` without checking the lock's value (`utility/jedis.go`), so if work outlives the 60-second expiry, a worker can delete another worker's lock. A MySQL row lock can't outlive its transaction, needs no extra service, and the guidelines favour fewer moving parts.

Algorithm: `billing.Run(ctx, asOf, dryRun)`

```
run := insert billing_runs(as_of_date, dry_run)
lastID := 0
loop:
    ids := SELECT id FROM subscriptions
           WHERE status = 'active' AND next_billing_date <= :asOf AND id > :lastID
           ORDER BY id LIMIT 100                                  -- keyset batches
    if empty: break
    lastID = max(ids)
    for each id:                                                  -- one transaction per subscription
        g.DB().Transaction(ctx, func(ctx, tx) error {
            sub := SELECT * FROM subscriptions
                   WHERE id = :id AND status = 'active' FOR UPDATE SKIP LOCKED   -- raw SQL
            if none: skipped (another worker has it); return nil
            periods := C.PeriodStart(k) for k from the period at next_billing_date,
                       while start <= asOf
                         and (expiry_date IS NULL or start <= expiry_date)
                         and (scheduled_close_date IS NULL or start < scheduled_close_date)
            existing := SELECT period_start FROM billing_records WHERE subscription_id = :id
            missing := periods − existing
            if len(missing) > 31: return safetyBoundError         -- logged, nothing written
            for each period in missing:
                base := plan item + recurring product items
                        + one-time items only if no billing_record exists yet
                rules := B.ApplicableDiscounts(ctx, tx, ...)      -- locks rule rows by ascending id
                items := B.PriceItems(base, rules, taxes)
                if dryRun: collect; continue
                INSERT invoices (draft, number NULL), invoice_items (base first, then linked)
                INSERT billing_records (period, invoice_id, inputs JSON)
                INSERT discount_applications; UPDATE discount_rules SET times_used = times_used + 1
                if settings.auto_confirm_recurring_invoices: confirm (number from A's sequence)
            UPDATE subscriptions SET next_billing_date = first period start > asOf
            A.audit.Log(...)
        })
        on error 1213/1205 (deadlock / lock wait): retry once
        on other error: count failed, append {subscription_id, message} to billing_runs.errors
finish run with counts
```

Why this is safe

Risk	Protection	Tested?
Run twice / retried after a crash	Missing-period diff + `UNIQUE (subscription_id, period_start)`	Duplicate period rejected (DB test)
Partially saved invoice	Invoice, items, billing record, discount use and date update in one transaction	— (unit test in Go)
Missed days (server down)	The loop bills every missing period up to today	—
Two runs at once	`FOR UPDATE SKIP LOCKED`	Second session skipped the locked row in 0.01 s
Discount over-used in a batch	Row lock + `CHECK (times_used <= max_uses)`	Over-use rejected (DB test)
Runaway billing from a bug	Safety bound of 31 periods per run	—
One bad subscription	Per-subscription transaction; logged, the rest continue	—

Scheduling

A nightly `gcron.AddSingleton` job calls, in order: C's `lifecycle.ActivateDue` → `billing.Run` → C's `lifecycle.CloseDue`. The **"Run billing now"** button runs the same sequence. With A's **test clock** in development, the demo can move to 1 April and bill a whole month in seconds.

Invoice lifecycle

Action	From → To	Rules
`confirm`	draft → confirmed	Number from `sequence.Next`; `confirmed_at`; `due_date` from payment terms; items become read-only; bulk confirm
`cancel`	draft/confirmed → cancelled	Only if `amount_paid = 0`; reason required; set `reversed_at` on its `discount_applications` and decrement `times_used` (UniBee does the same with a "rollback" status on discount usage)
(automatic)	confirmed → paid	When `amount_paid = total` after a payment
`send`	confirmed or paid	Enqueue an `email` job with dedupe key `invoice-sent:{id}`, PDF attached; `sent_at`
`pdf`	any	`github.com/go-pdf/fpdf` (UniBee uses it for invoices): company header, items grouped by rule/rate, totals

A cancelled invoice keeps its billing record, so the period isn't re-billed automatically.

Recording a payment (one transaction)

1. `SELECT ... FROM invoices WHERE id = ? FOR UPDATE`
2. Status must be `confirmed`; `0 < amount <= total − amount_paid` (clear message with the balance)
3. `INSERT payments` with the form's `idempotency_key`. On error **1062** for that key, return the payment already recorded with it (HTTP 200): the double submit becomes harmless.
4. `amount_paid += amount`; if equal to `total`, `status = 'paid'`
5. `audit.Log`

Overdue reminders (should-have)

A daily job enqueues email reminders for confirmed invoices 1, 7 and 15 days past due, with dedupe keys `overdue:{invoice}:{day}`, so a re-run never sends twice. This is a light version of Kill Bill's overdue module, which moves accounts through configurable overdue states.

---

4. Reporting definitions (write these into the README)

Metric	Definition	Measured on 10k subs / 100k invoices
Active subscriptions + **MRR**	`COUNT(*)`, `SUM(recurring_monthly)` where `status = 'active'`	19 ms
Revenue received (period)	`SUM(payments.amount)` by `payment_date`	30 ms
Outstanding	`SUM(total − amount_paid)` where `status = 'confirmed'`	99 ms
Overdue	`status = 'confirmed' AND due_date < today AND amount_paid < total`; list page 1 / count	8 ms / 60 ms
Customer invoice list	by `customer_id`, newest first, 25 rows	11 ms
ARR	MRR × 12	—
Churn (period)	Subscriptions closed in the period whose close reason has `is_churn = TRUE`, by reason; churned MRR	—

Timings were measured in this sandbox with MySQL 8.0.46 on seeded data, client start-up included; repeat them on your own machine in Phase 4. All aggregates run in SQL; every list is paginated.

---

5. API endpoints you own

Method	Route	Access	Notes
POST	`/api/billing/runs`	Admin, Staff	`{ asOfDate?, dryRun? }`; returns the run summary
GET	`/api/billing/runs`	Admin, Staff	History with errors
GET	`/api/subscriptions/{id}/upcoming-invoice`	Admin, Staff, owning Customer	Dry-run preview of the next invoice
GET	`/api/invoices`	Admin, Staff	Filters: status, payment status, customer, subscription, dates, overdue; paginated
GET	`/api/invoices/{id}`	Admin, Staff	Items (linked), payments, balance
POST	`/api/invoices/{id}/actions/{confirm|cancel|send}`	Admin, Staff	
POST	`/api/invoices/actions/bulk-confirm`	Admin, Staff	Per-id result
GET	`/api/invoices/{id}/pdf`	Admin, Staff, owning Customer	
POST	`/api/invoices/{id}/payments`	Admin, Staff	`{ method, amount, paymentDate, note, idempotencyKey }`
GET	`/api/payments`	Admin, Staff	Filters: dates, method, customer
GET	`/api/reports/summary`, `/api/reports/overdue`, `/api/reports/revenue-by-month`, `/api/reports/mrr`	Admin, Staff	
GET	`/api/portal/invoices[/{id}]`	Customer	Own only, else 404

---

6. Frontend you own

- **Billing:** "Run billing now" with date and dry-run switch, run summary (due, created, skipped, failed with reasons), run history.
- **Invoices:** list with status and payment-status tags, filters, bulk confirm; detail with items grouped (base → its discount → its tax), payments and balance; actions; "Record payment" modal using A's `useIdempotencyKey()`.
- **Upcoming invoice** card on C's subscription detail page.
- **Finance dashboard:** AntD `Statistic` cards (MRR, active subscriptions, revenue, outstanding, overdue), date filter, overdue table, revenue-by-month chart, MRR by plan, churn by reason; each card links to its filtered list.
- **Customer tab** and **portal "My Invoices"** with PDF download.

---

7. What you depend on from others

From	What	When
A	`clock.Today`, `sequence.Next`, `jobs.Enqueue`, `audit.Log`, settings, `auth.CustomerID`, idempotency hook	Phase 1
B	`ApplicableDiscounts`, `PriceItems`, `TaxRulesFor`	Stub Phase 1, real Phase 2
C	`PeriodStart` / `PeriodEnd`, `lifecycle.ActivateDue` / `CloseDue`, `recurring_monthly`, close reasons	Stub Phase 1, real Phase 2

You are the most downstream member: push for stubs early and build against them.

---

8. Phase-by-phase checklist

Phase 0 — Kickoff (with team)

- [ ] Walk everyone through the billing algorithm and the safety table

Phase 1 — Foundation

- [ ] Repositories for your tables; seed script skeleton

Phase 2 — Core workflows

- [ ] `billing.Run` with per-subscription transactions, diff, safety bound, dry run, run log
- [ ] Invoice actions, PDF, payments with idempotency
- [ ] Invoice and billing screens
- [ ] **Checkpoint 2 (team):** quote → active subscription → correct invoice with linked items

Phase 3 — Integration

- [ ] Nightly scheduler calling C's steps in order; upcoming-invoice preview
- [ ] Dashboard, portal invoices, send via job queue + MailHog, overdue reminders

Phase 4 — Testing and hardening

- [ ] Tests in section 9
- [ ] Seed 10,000 subscriptions and 100,000 invoices; time every page; `EXPLAIN` slow queries; record the table for reviewers

Phase 5 — Demo prep

- [ ] Demo data with due subscriptions, overdue invoices and partial payments

---

9. Tests and edge cases you must cover

- Run billing twice for the same date → 0 created, all skipped
- Crash after inserting items (inject an error) → nothing saved for that subscription; re-run creates it once
- Billing not run for 3 days on a daily plan → 3 invoices on the next run
- 33 missing periods → safety bound refuses and logs; nothing written
- 300 subscriptions due on one day → all billed; time recorded
- Two runs started together → each subscription billed once
- Discount with 1 use left and 5 eligible subscriptions → applied exactly once
- One-time product → first invoice only; paused, closed, future-start and end-of-term-closing subscriptions → correct periods only
- Items: discount and tax linked to the right base item; totals equal B's preview to the cent
- Draft has no number; confirming three invoices gives consecutive numbers; a rolled-back confirm consumes no number
- Payment over balance, on a draft, on a cancelled invoice → rejected; same idempotency key twice → one payment
- Partial then full payment → `partial` then `paid`; cancel with payments → rejected; cancel releases the discount use
- Overdue appears the day after `due_date`, disappears when fully paid; reminders never sent twice
- Customer A fetching customer B's invoice or PDF → 404

---

10. Git branches (suggested)

`feature/invoice-items`, `feature/billing-engine`, `feature/invoice-actions-pdf`, `feature/payments`, `feature/dashboard`, `feature/portal-invoices`, `feature/overdue-reminders`, `feature/seed-and-perf`

---

11. Your demo segment

1. Open a subscription's **upcoming invoice** preview.
2. Move the test clock to the billing date; click **Run billing now** → invoices with linked discount and tax items.
3. Click it again → 0 created, all skipped.
4. Confirm an invoice (number assigned now), record a partial payment, submit the same form twice (one payment), then pay the rest → Paid.
5. Open the dashboard on the large dataset: MRR, revenue, outstanding, overdue, churn, all well under two seconds.

---

12. Questions you should be ready to answer

- Exactly how do you guarantee no duplicate invoices and no skipped periods? How is that different from UniBee and Kill Bill?
- Why a MySQL row lock instead of a Redis lock?
- What does the safety bound protect against?
- Why are invoice items linked, and why are they never updated?
- Why is the invoice number assigned on confirm?
- How did you verify the two-second target?
- How does the full flow work from signup to dashboard? (Explain A, B and C at a high level.)